Introduction to Programming

W12 Recursion

Stephan Krusche

14 January 2026
Technical University of Munich

Schedule

#	Date	Subject
1	15.10.25	Introduction
2	22.10.25	Control Flow in Programming
3	29.10.25	**In-Depth Core Concepts***
4	05.11.25	Core Data Structures
5	12.11.25	Code Reuse and Structure
6	19.11.25	Type Flexibility and Safety
7	26.11.25	**In-Depth Object Orientation***
8	03.12.25	Functional Programming Essentials
9	10.12.25	Algorithms and Data Handling
10	17.12.25	Programming Languages
11	07.01.26	Graphical User Interfaces
12	**14.01.26**	**Recursion**
13	21.01.26	Concurrency
14	28.01.26	Beyond Programming
15	04.02.26	Course Review

*** Repetition**

Roadmap

- **Context**
  - Apply OOP concepts: abstraction, encapsulation, inheritance and polymorphism
  - Use control structures and data types
  - Use enums, annotations, generics, collections, iterators, lambda expressions, and streams
  - Apply error handling, implement algorithms, and understand programming languages
  - Implement GUIs using JavaFX
- **Learning goals**
  - Apply direct and indirect recursion
  - Understand the terminating conditions of recursion
  - Implement algorithms based on **recursion** (and iteration)
  - Explain the difference between recursion and iteration
  - Transform algorithms: iteration <—> recursion

Outline

- The principle of recursion
- Example: Towers of Hanoi
- Example: Binary search
- Example: Koch snowflake
- Example: Mergesort

Divide and conquer

- Basic, yet very useful principle of algorithms
- To solve a problem
  1. Divide it into several subproblems
  2. Solve the individual subproblems
  3. Combine the partial solutions to a total solution

Recursion

- Principle of recursion: run the same calculation pattern over and over again with simpler or smaller input data until you arrive at a trivial input
- **Realization:** methods that call themselves directly or indirectly
- **Recursion** is the standard implementation of the divide and conquer principle

Recursive methods

- **Definition**: a method `f` is called (directly) recursive if calls to `f` occur in the body of `f`
- The method `f` is called (indirectly) recursive if in the body of `f` a method `g` is called, which in turn leads directly or indirectly to calls of `f`
- For each recursive call, a new instance of the respective method is started

→ Each instance has its own local variables and parameters, which are not visible from outside

Example: factorial function

- The factorial function **`n!`** calculates the product of the numbers 1, 2, …, n
- Recursively, **`n!`** can therefore be calculated as follows

$$0! = 1, \quad n! = n \cdot (n-1)! \quad (\text{for } n \geq 1)$$

Java code

```
public static long fac(long n) {
    if (n > 0) {
        return n * fac (n - 1);
    } else {
        return 1;
    }
}
```

Direct recursive method call

Important: a terminating condition without recursive function call to prevent endless program

What happens when calling **`fac`**

```
fac(4)
= ( if(4>0){ 4*fac(4-1) } else { 1 } )
= 4*fac(3) =4*( if(3>0){ 3*fac(3-1) } else{ 1 } )
= 4*3*fac(2)
= 4*3*( if (2>0) { 2*fac(2-1) } else { 1 } )
= 4*3*2*fac(1)
= 4*3*2*( if (1>0) { 1*fac(1-1) } else { 1 } )
= 4*3*2*1*fac(0)
= 4*3*2*1*( if (0>0) { 0*fac(0-1) } else { 1 } )
= 4*3*2*1*1
```

Example: wrong terminating condition

```
public static long fac(long n) {
    if (n == 100) {
        return 1;
    } else {
        return n * fac (n - 1);
    }
}
```

When you invoke this method with **n < 100**, it will not terminate

```
Exception in thread "main" java.lang.StackOverflowError
```

Example: greatest common divisor

```
int gcd(int n, int m) {
    if (n == m) {
        return n;
    } else if (n < m) {
        return gcd(n, m - n);
    } else {
        return gcd(n - m, m);
    }
}
```

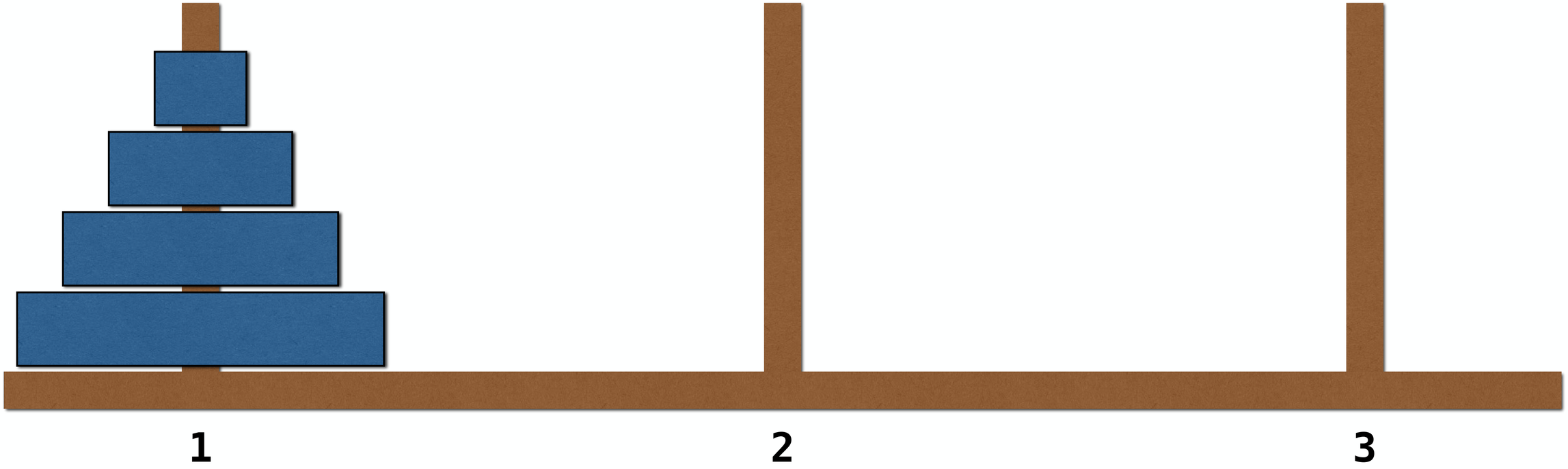
Important: a terminating condition without recursion to prevent endless program

Direct recursive method calls

Outline

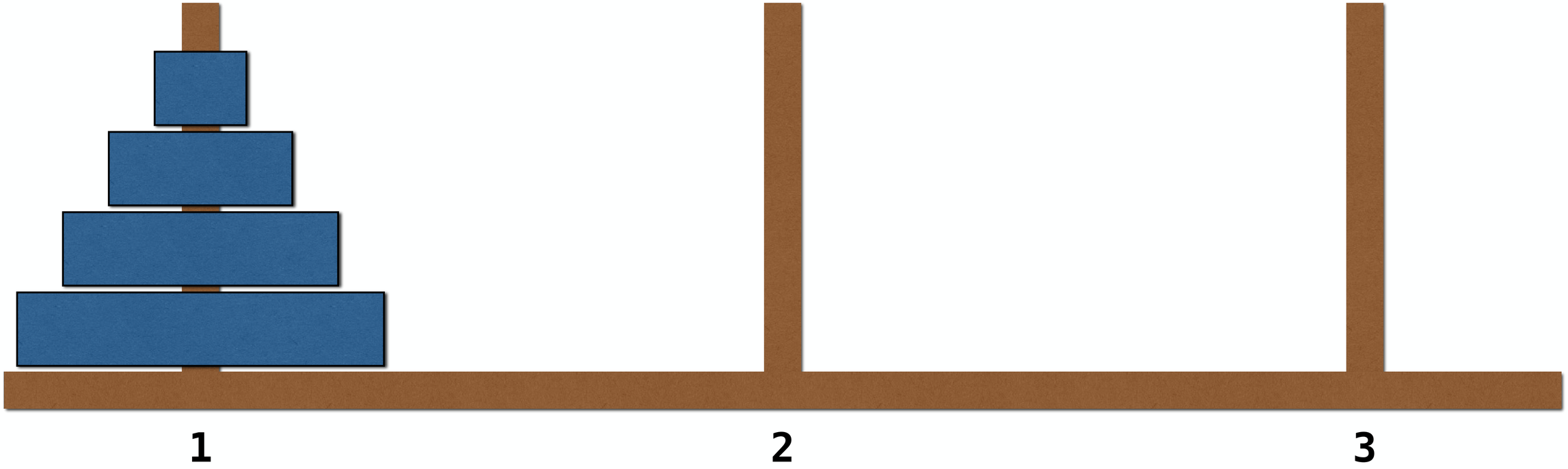
- The principle of recursion
- Example: Towers of Hanoi
- Example: Binary search
- Example: Koch snowflake
- Example: Mergesort

Problem

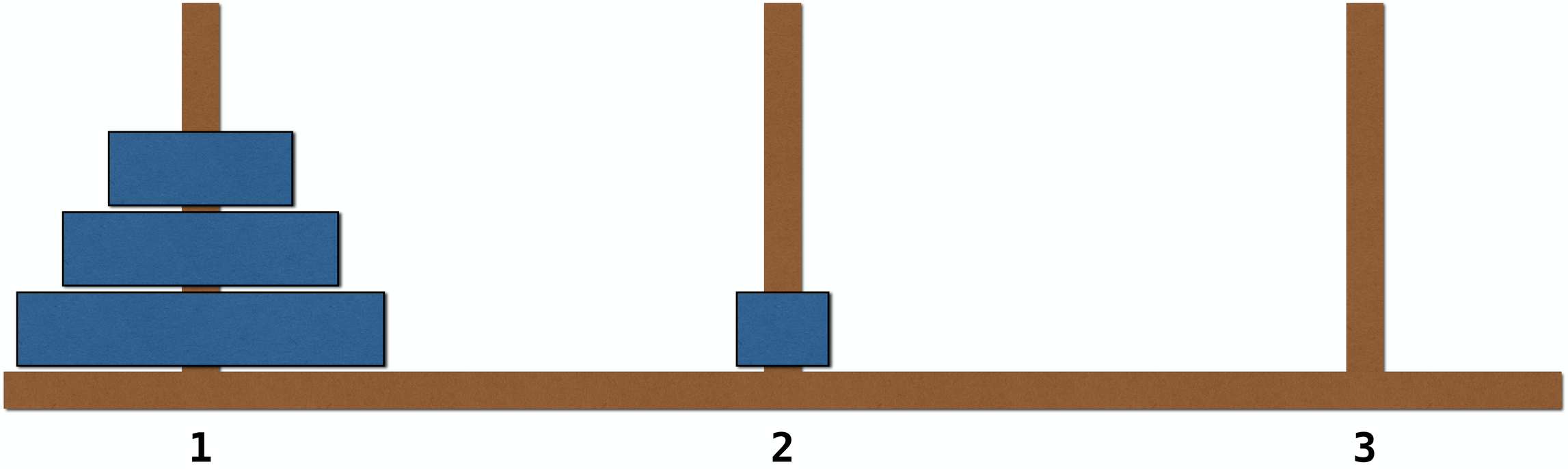


- Move the stack from left to right
- You may move exactly one ring in each turn
- A larger ring may never be placed on top of a smaller one

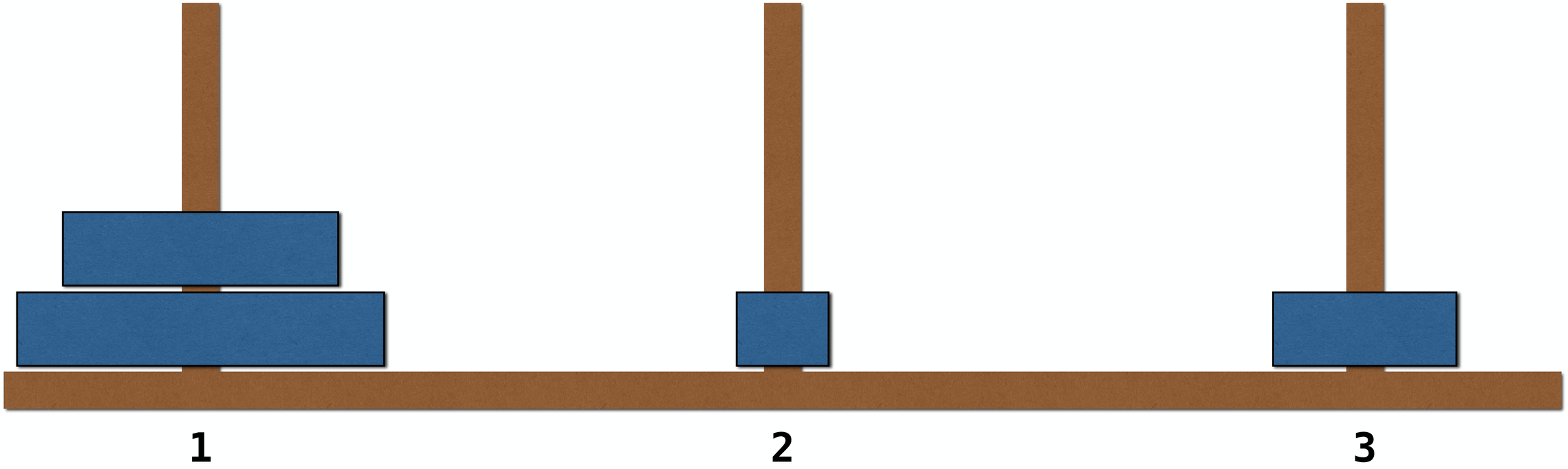
Move slices



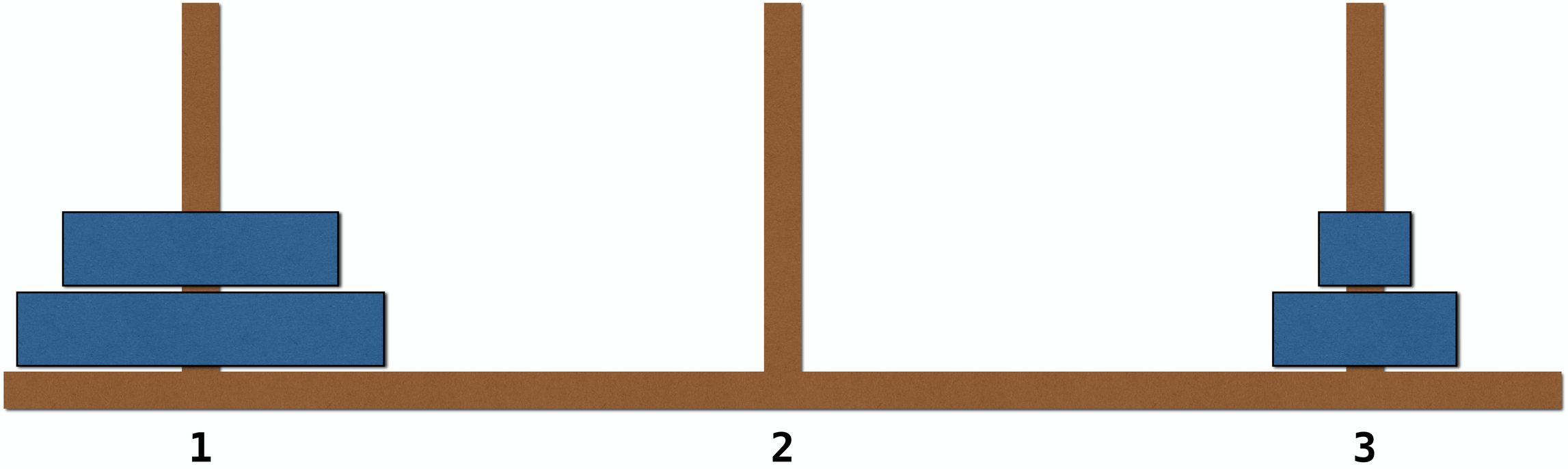
Move slices



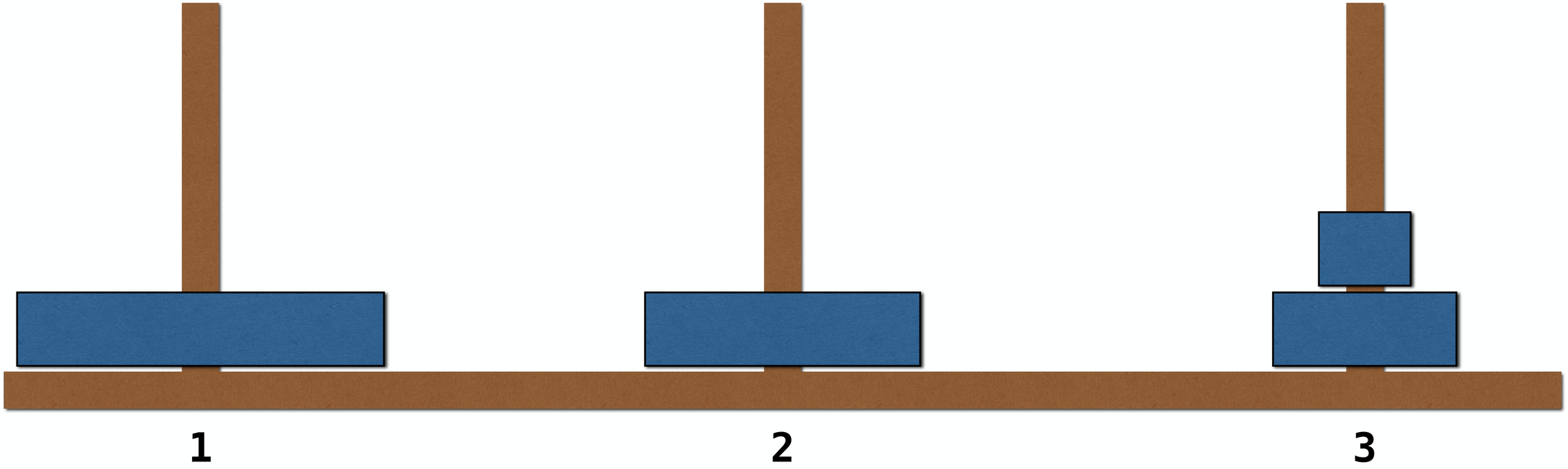
Move slices



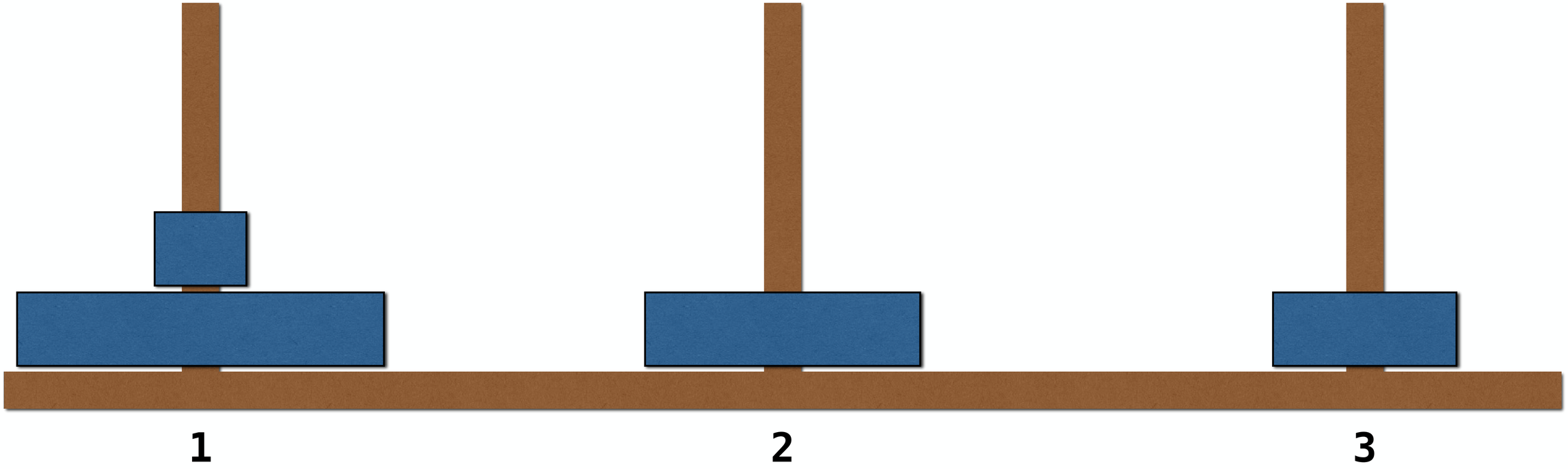
Move slices



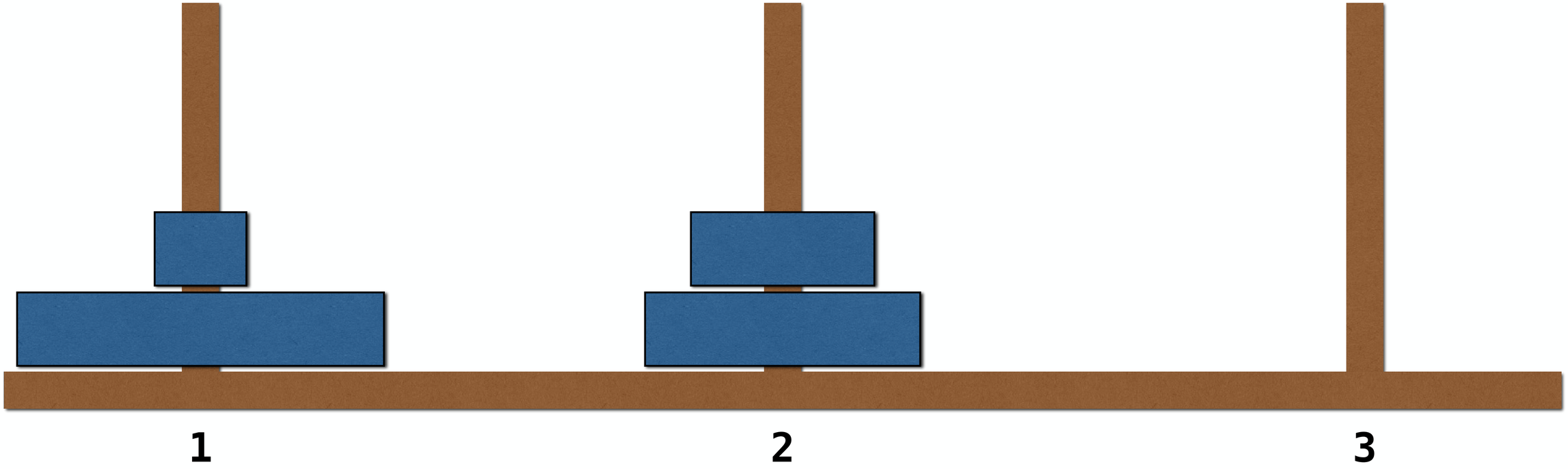
Move slices



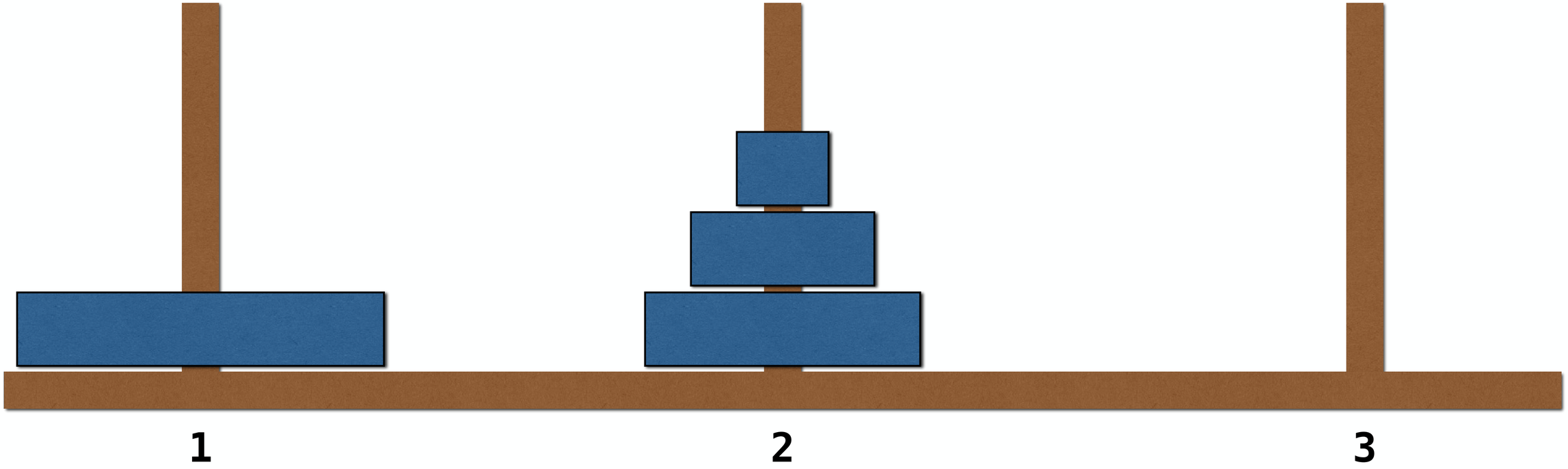
Move slices



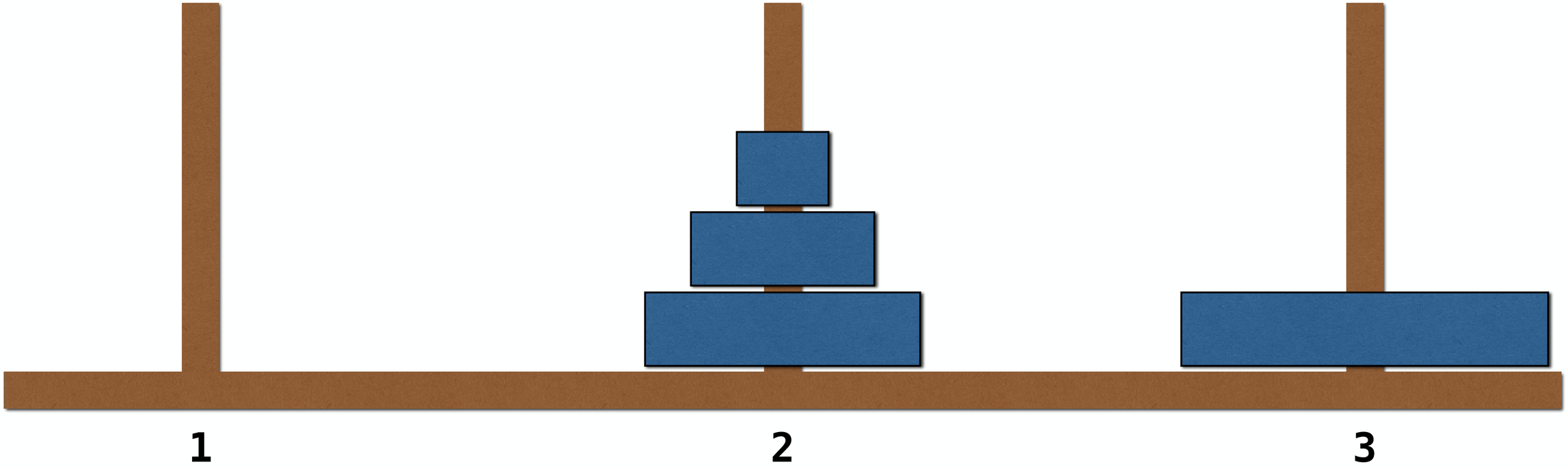
Move slices



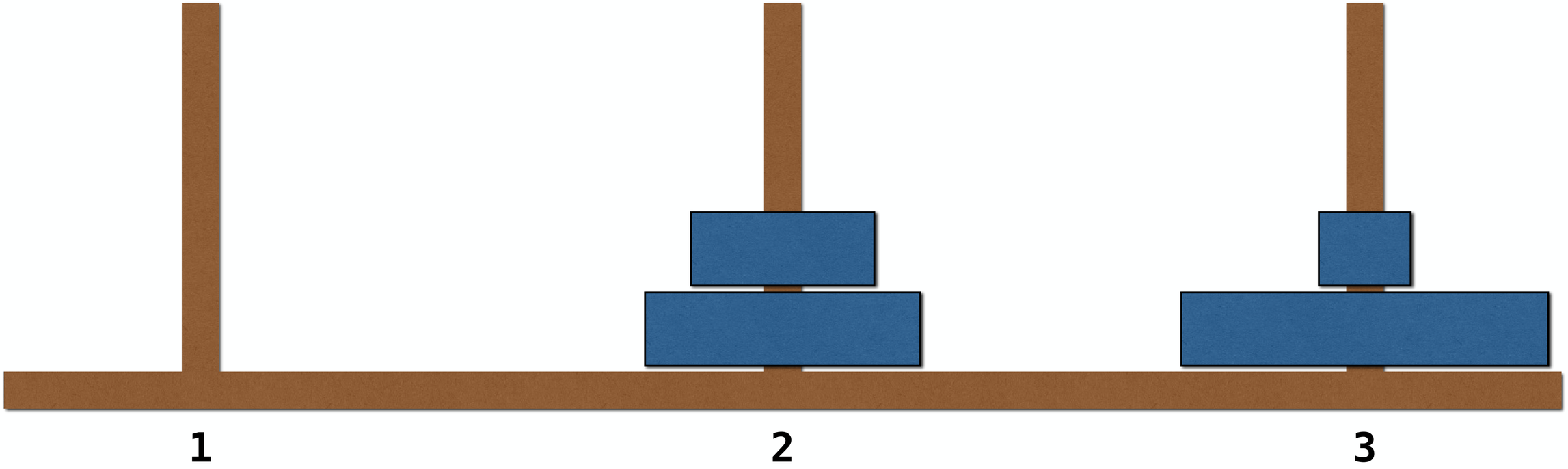
Move slices



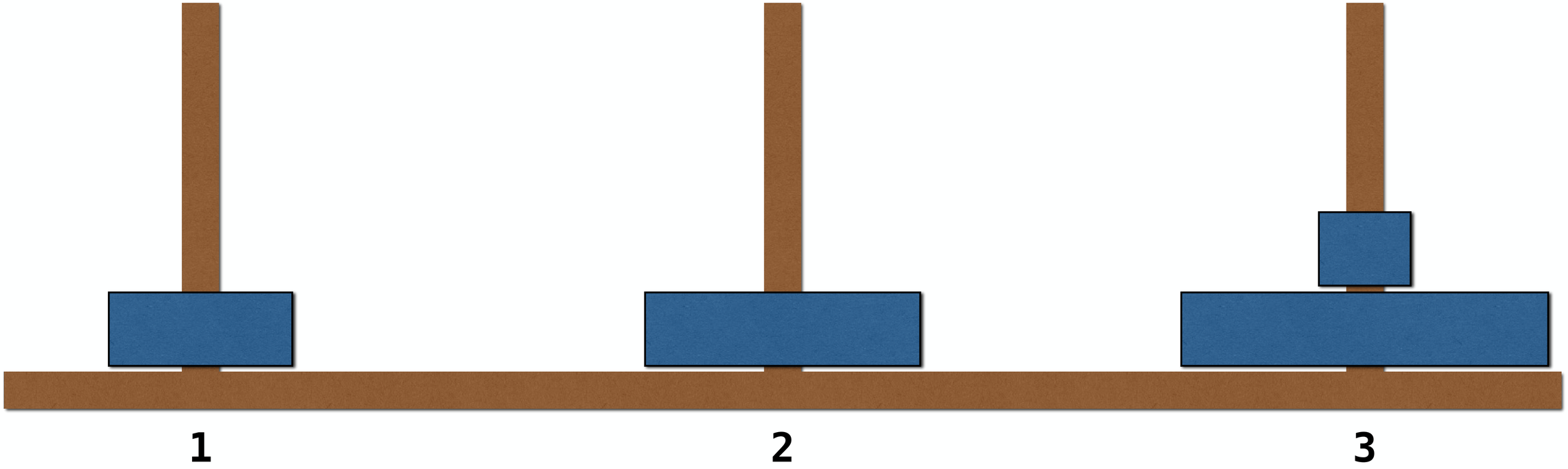
Move slices



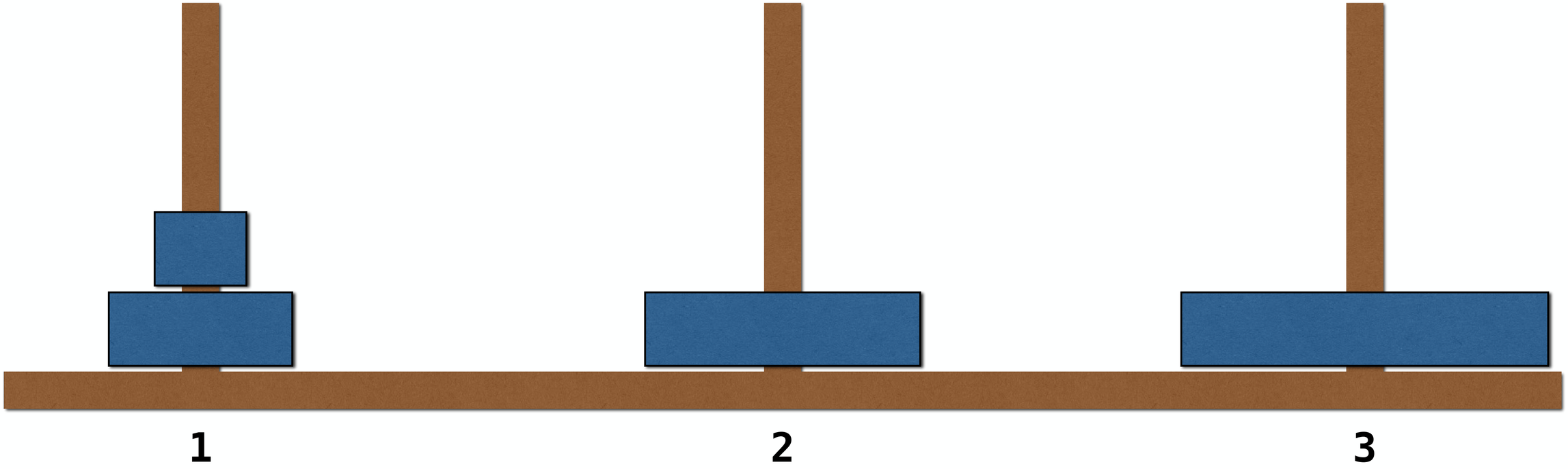
Move slices



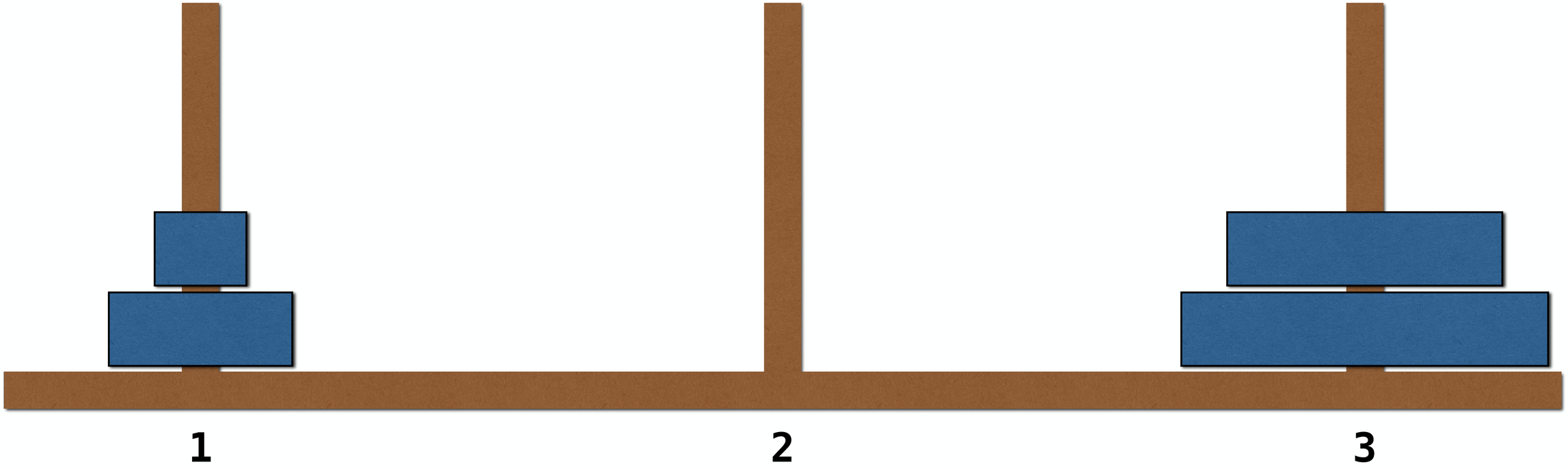
Move slices



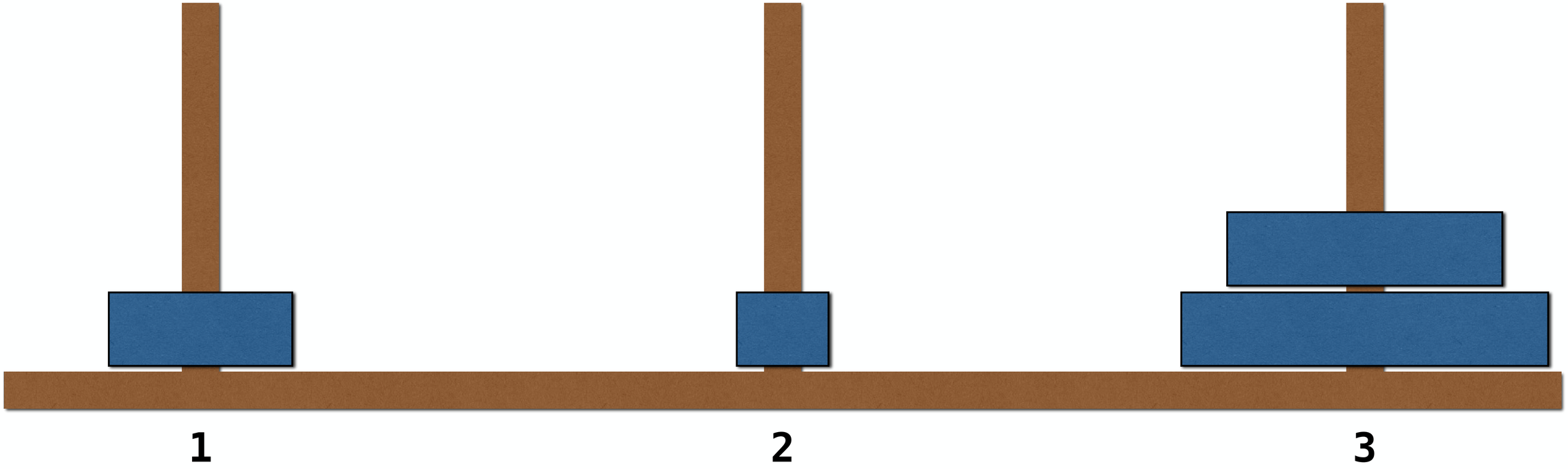
Move slices



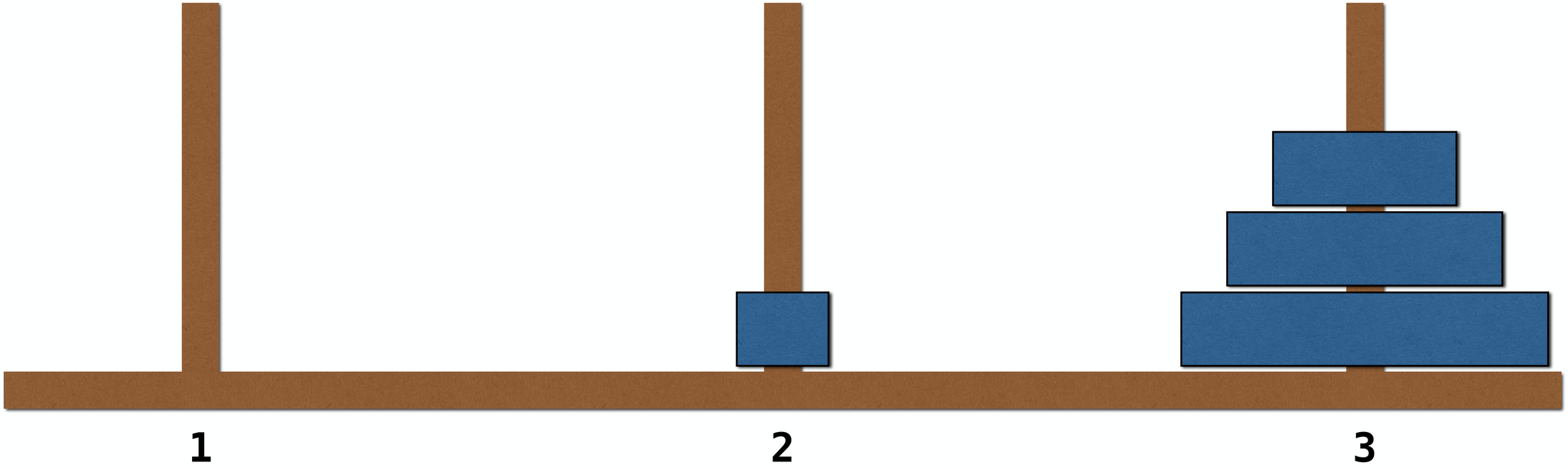
Move slices



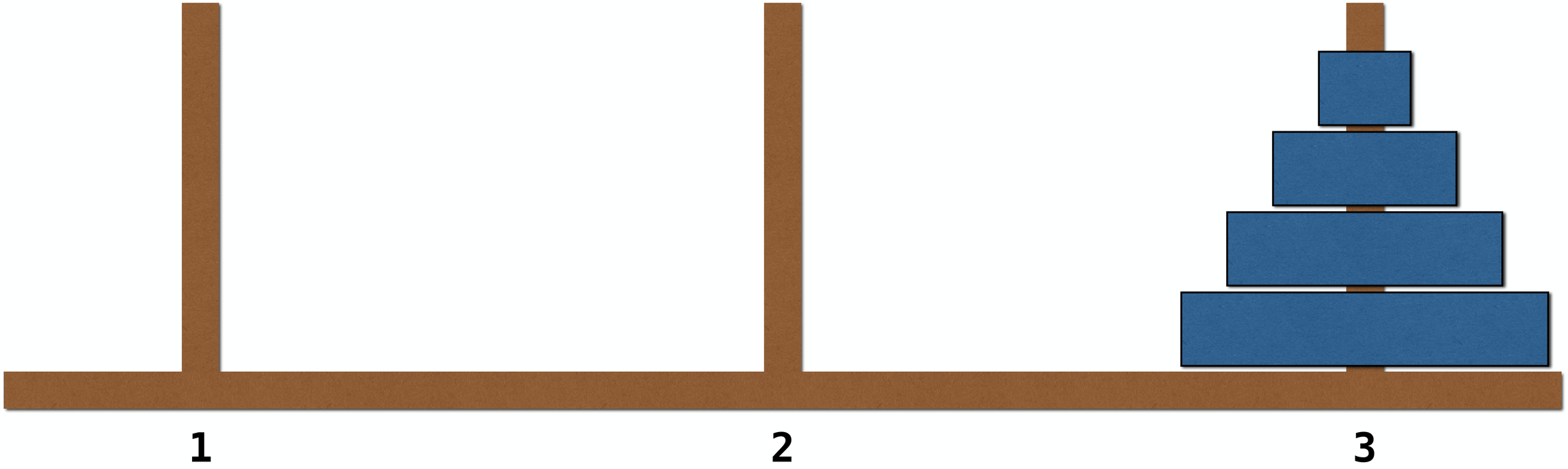
Move slices



Move slices

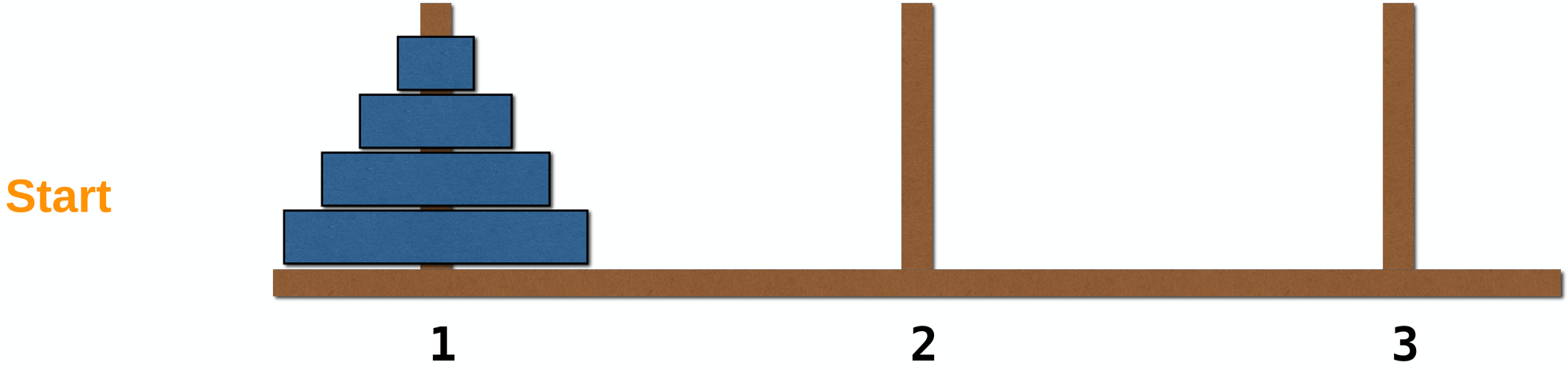


Move slices



Idea

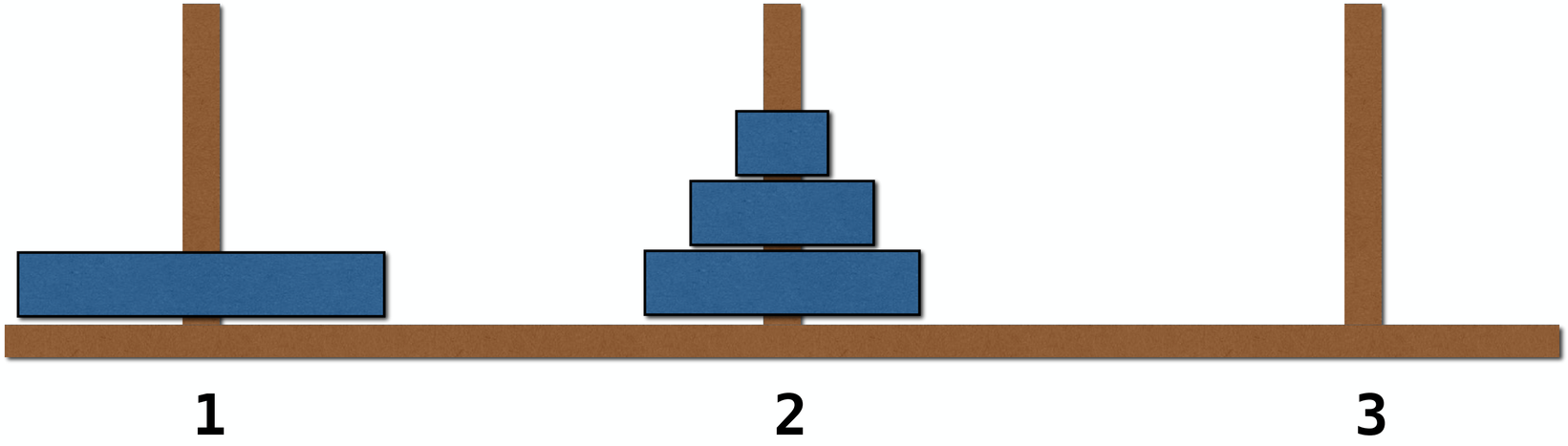
- Moving a tower of height `h = 0` is simple: nothing to do
- Moving a tower of height `h > 0` from position `a` to position `b` is decomposed into three subtasks
  1. Move the top `h - 1` slice to the free place (using recursion)
  2. Move the lowest slice to the target position
  3. Move the intermediate slices to the target position (using recursion)
- Moving a tower of height `h > 0` thus requires moving a tower of height `h - 1` twice



Idea

- Moving a tower of height `h = 0` is simple: nothing to do
- Moving a tower of height `h > 0` from position `a` to position `b` is decomposed into three subtasks
  1. **Move the top `h - 1` slice to the free place (using recursion)**
  2. Move the lowest slice to the target position
  3. Move the intermediate slices to the target position (using recursion)
- Moving a tower of height `h > 0` thus requires moving a tower of height `h - 1` twice

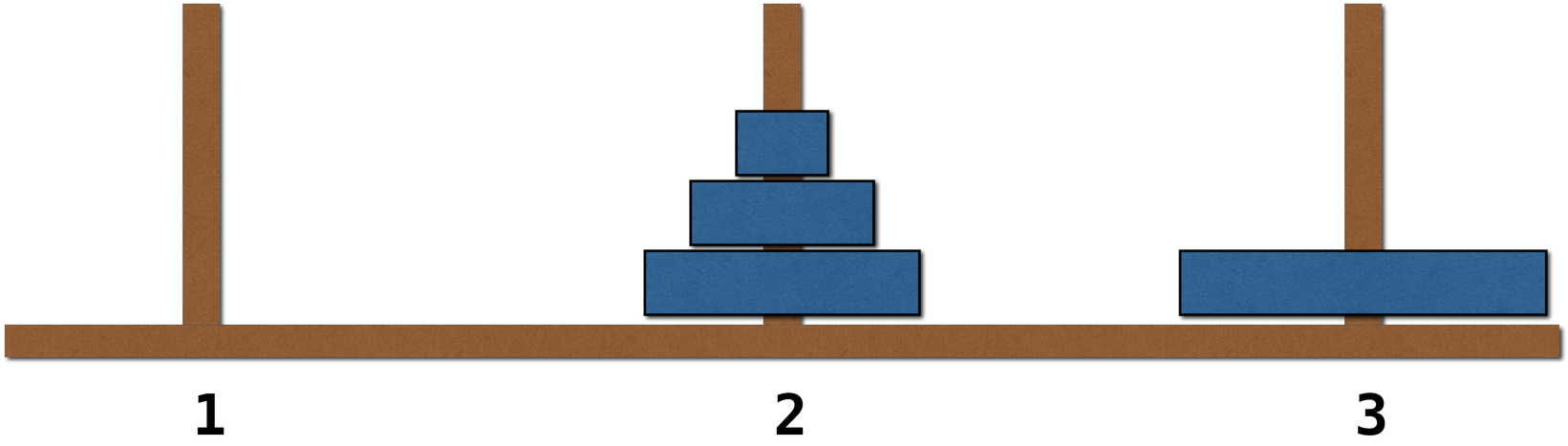
After step 1



Idea

- Moving a tower of height `h = 0` is simple: nothing to do
- Moving a tower of height `h > 0` from position `a` to position `b` is decomposed into three subtasks
  1. Move the top `h - 1` slice to the free place (using recursion)
  2. **Move the lowest slice to the target position**
  3. Move the intermediate slices to the target position (using recursion)
- Moving a tower of height `h > 0` thus requires moving a tower of height `h - 1` twice

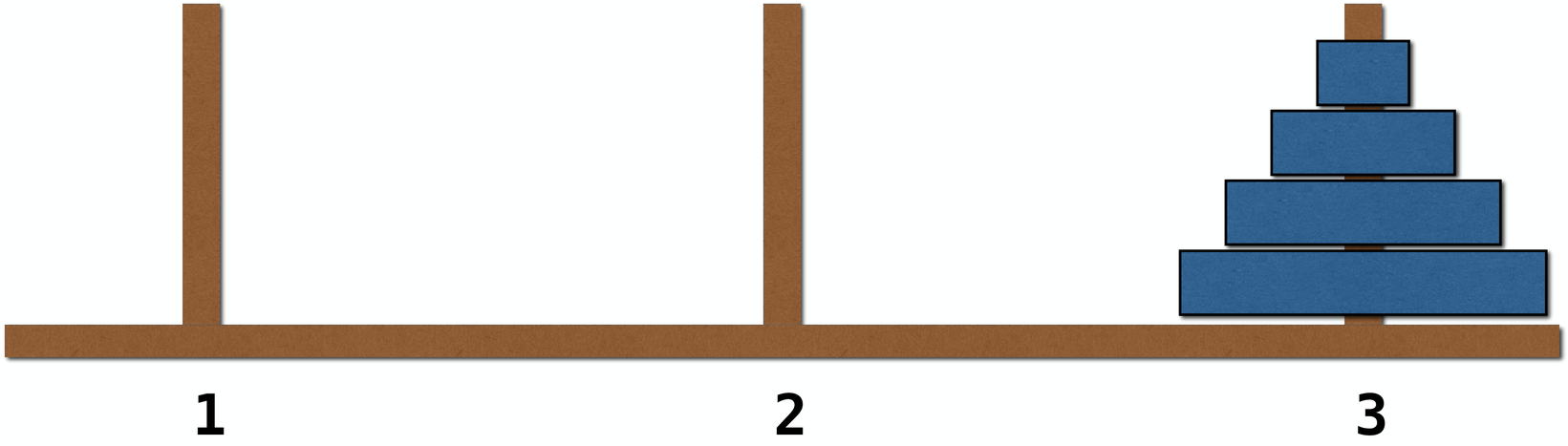
After step 2



Idea

- Moving a tower of height `h = 0` is simple: nothing to do
- Moving a tower of height `h > 0` from position `a` to position `b` is decomposed into three subtasks
  1. Move the top `h - 1` slice to the free place (using recursion)
  2. Move the lowest slice to the target position
  3. **Move the intermediate slices to the target position (using recursion)**
- Moving a tower of height `h > 0` thus requires moving a tower of height `h - 1` twice

After step 3



Code

Input: spot 1, 2, or 3 where **a** and **b** must be different

Direct recursive method calls

```
public static void move(int height, int a, int b) {
    if (height > 0) {
        int c = free(a, b);      // c is the free spot
        move(height - 1, a, c); // move height - 1 slices from spot a to spot c
        // then the remaining slice from a to b
        System.out.println("move " + a + " to " + b);
        move(height - 1, c, b); // move height - 1 slices from spot c to spot b
    }
}
```

Remains to determine the free space ...

Free place

	1	2	3
1		3	2
2	3		1
3	2	1	

(read: **`free(x,y)`** is the position which is not occupied)

Apparently, the result depends only on the **sum** of the two arguments ...

	1	2	3
1		3	4
2	3		5
3	4	5	

Implementation

- We implement the method `free(...)` with `switch`

```
public static int free(int a, int b) {
    switch (a + b) {
        case 3:  return 3;
        case 4:  return 2;
        case 5:  return 1;
        default: return -1;
    }
}
```

Shorter alternative (more difficult to understand)

```
public static int free(int a, int b) {
    return 6 - (a + b);
}
```

Tower of Hanoi

```java
public class Hanoi {
    public static void move(int height, int a, int b) {
        if (height > 0) {
            int c = free(a, b);
            move(height - 1, a, c);
            System.out.println("move " + a + " to " + b);
            move(height - 1, c, b);
        }
    }

    public static int free(int a, int b) {
        return 6 - (a + b);
    }

    public static void main(String[] args) {
        int h = InputReader.readInt("Height of the tower: ");
        System.out.println("");
        move (h, 1, 3);
        System.out.println("");
    }
}
```

W12E02 - Recursion sum

Start exercise

Easy

Not started yet.

Due by tonight

10 min

3 pts

- **Problem statement**
  - Create a recursive function that determines the sum of integers **`from 1 to the value n`** (input parameter)
  - Implement the same function using iteration (e.g. using a for loop)

Outline

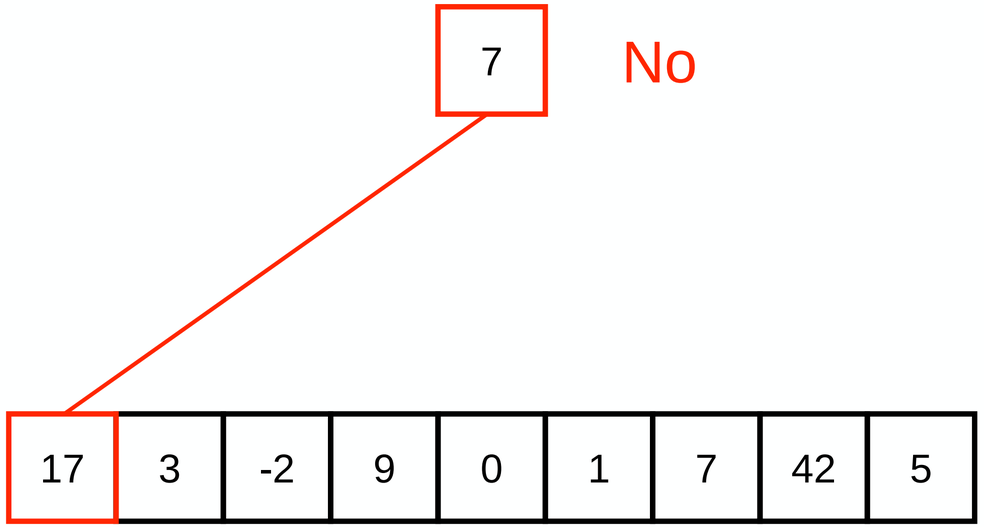
- The principle of recursion
- Example: Towers of Hanoi
- Example: Binary search
- Example: Koch snowflake
- Example: Mergesort

Binary search

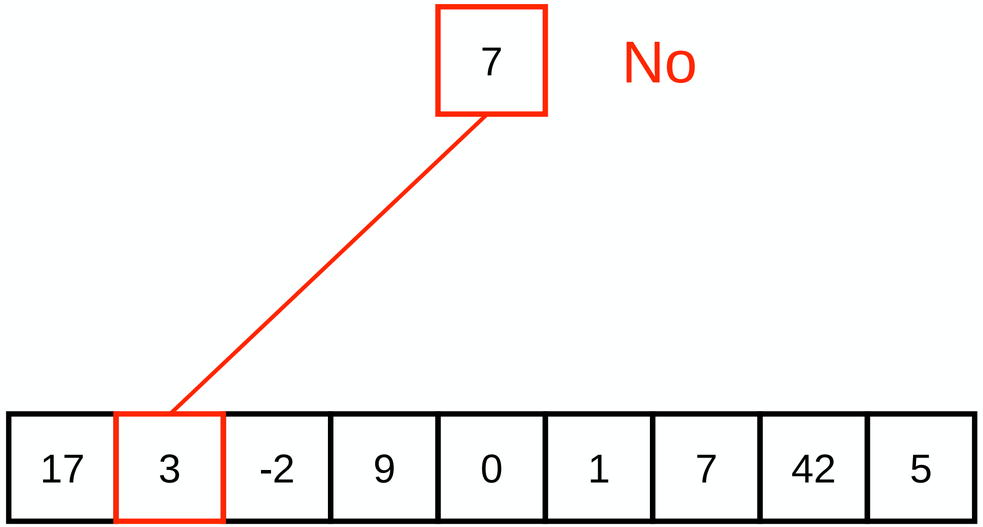
- Suppose we want to find out whether the element **`x = 7`** is contained in an **`array`**
- Naive approach
  - Compare **`7`** in order with the elements **`array[0], array[1],`** etc.
  - If there is an **`index`** with **`array[index] == 7`**, output **`index`**
  - Otherwise, output **`-1`** which means “not found"

```
public static int find(int[] array, int x) {
    int index = 0;
    while (index < array.length && array[index] != x) {
        index++;
    }
    if (index == array.length) {
        return -1;
    }
    else {
        return index;
    }
}
```

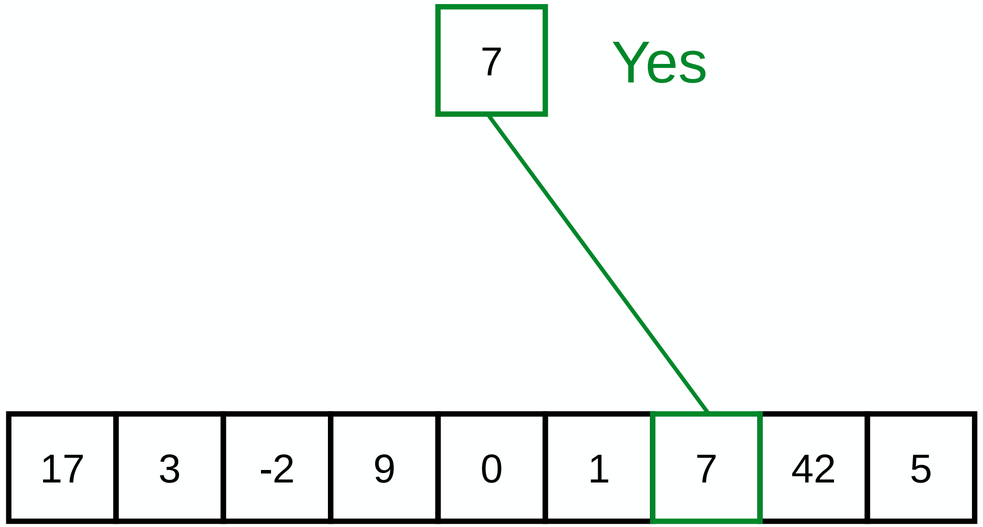
Naive search



Naive search



Naive search



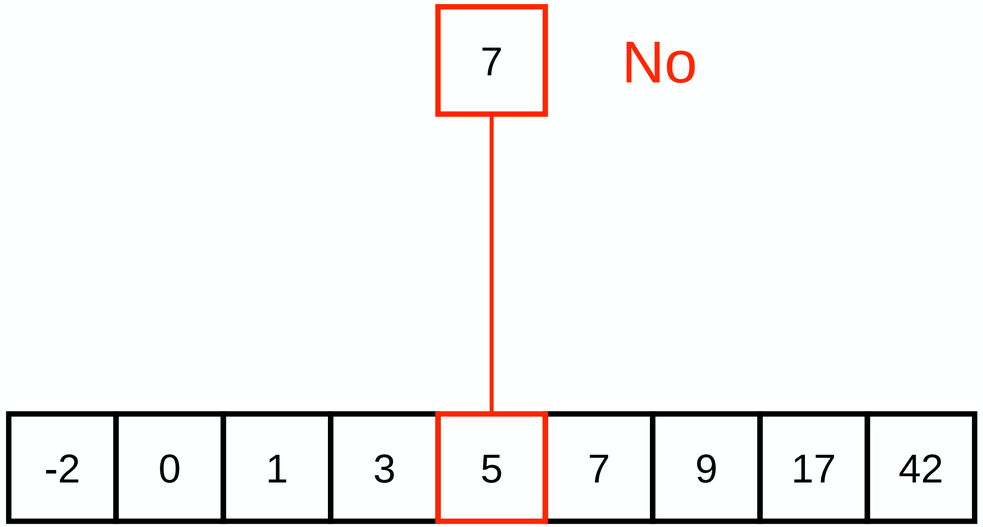
- Analysis: in the example, `7` comparisons are needed
- In the worst case, `n` comparisons are necessary for a field of length `n`
- If `7` actually occurs in the field, we need on average `(n + 1)/2` many comparisons
- Can we do it better?

Idea

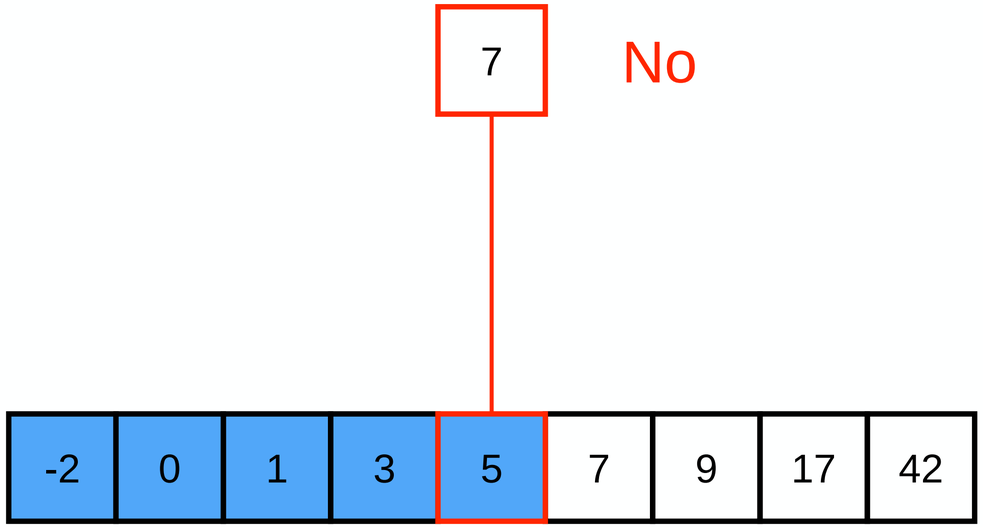
- Sort the field
- Compare **7** with the value in the middle
- if it's equal or if its elements match, the search is complete
- If **7** is smaller, we only need to continue searching on the left
- If **7** is greater, we only need to continue searching on the right

→ **Binary** search

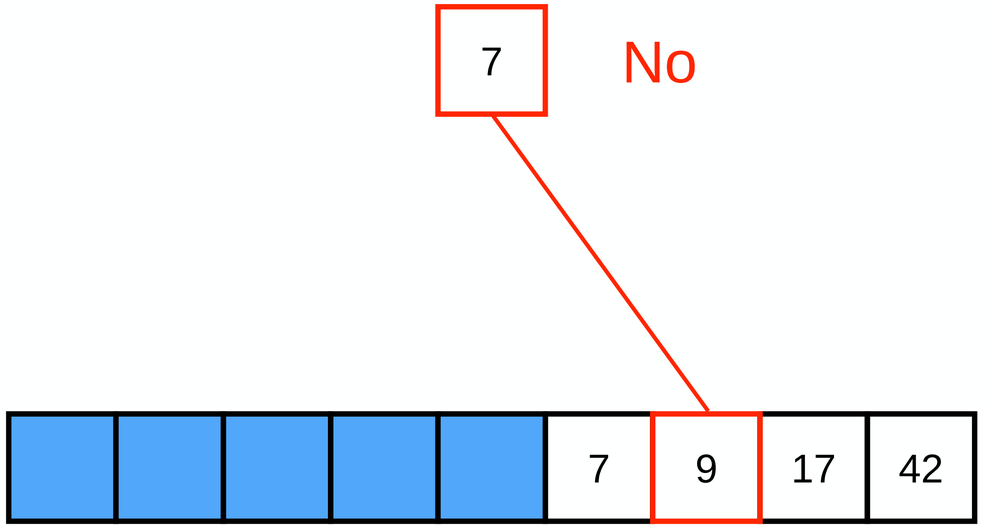
Binary search



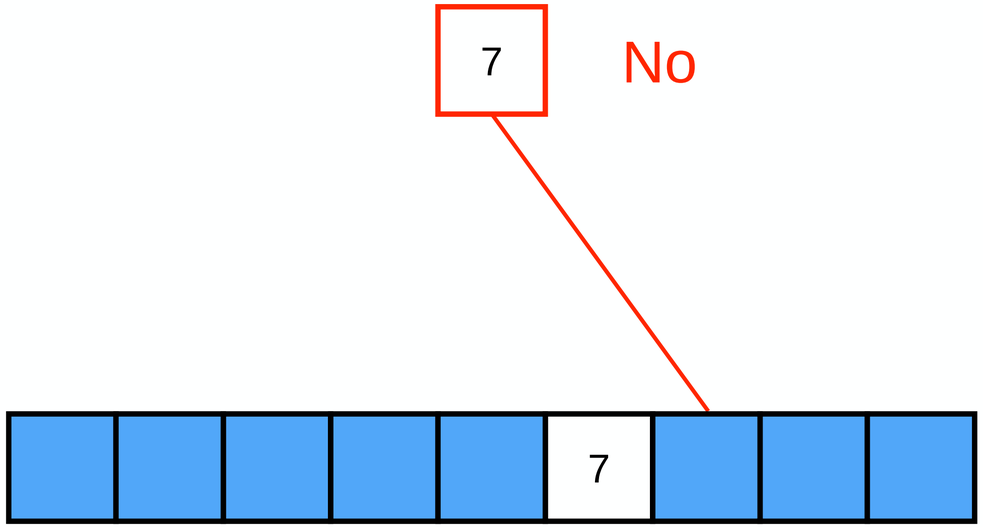
Binary search



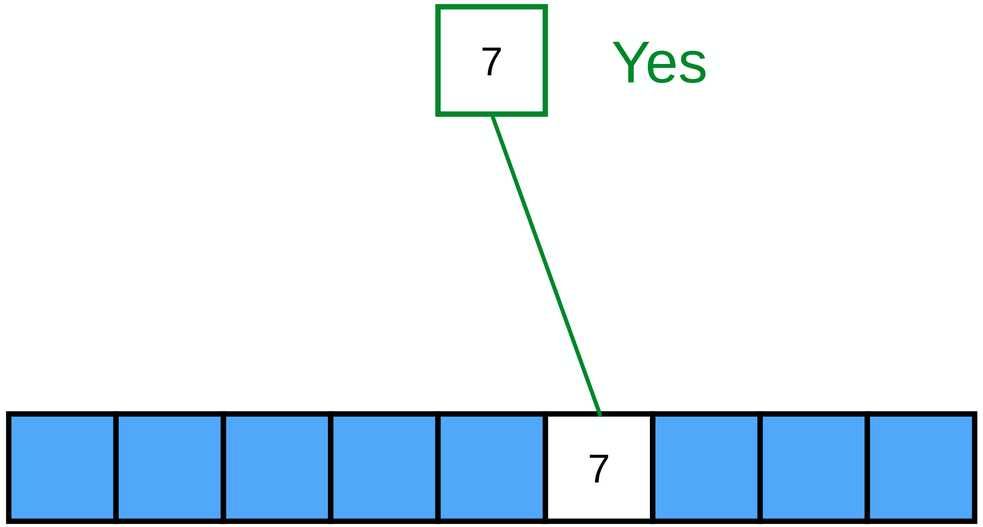
Binary search



Binary search



Binary search



- Analysis: binary search only needs **3** comparisons
- If the sorted array has $\mathbf{2^n-1}$ elements, at most **n** comparisons are necessary

Idea

- We introduce an auxiliary function that searches in the interval `[startIndex, endIndex]`

```
public static int find(int[] array, int element, int startIndex, int endIndex)
```

Then, we can invoke this method

```
public static int find(int[] array, int element) {
    return find(array, element, 0, array.length - 1);
}
```

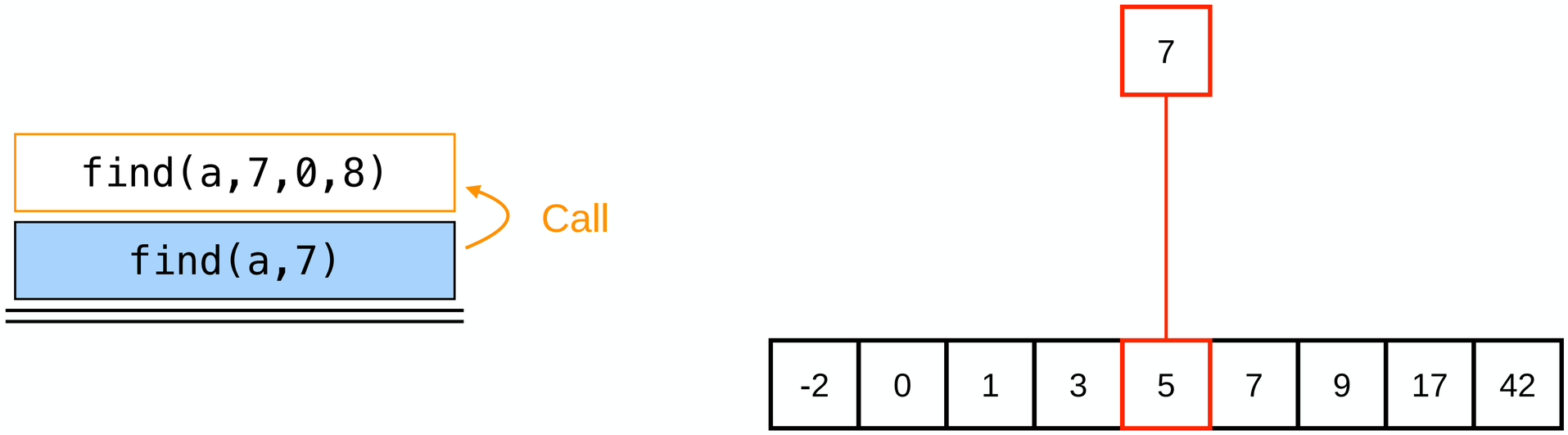
Implementation of binary search

```
public static int find(int[] array, int element, int startIndex, int endIndex) {
    int middle = (startIndex + endIndex) / 2;
    if (array[middle] == element) {
        return middle; // found
    }
    else if (startIndex == endIndex) {
        return -1; // not found
    }
    else if (element > array[middle]) {
        return find(array, element, middle + 1, endIndex);
    } else if (startIndex < middle) {
        return find(array, element, startIndex, middle - 1);
    } else {
        return -1; // not found
    }
}
```

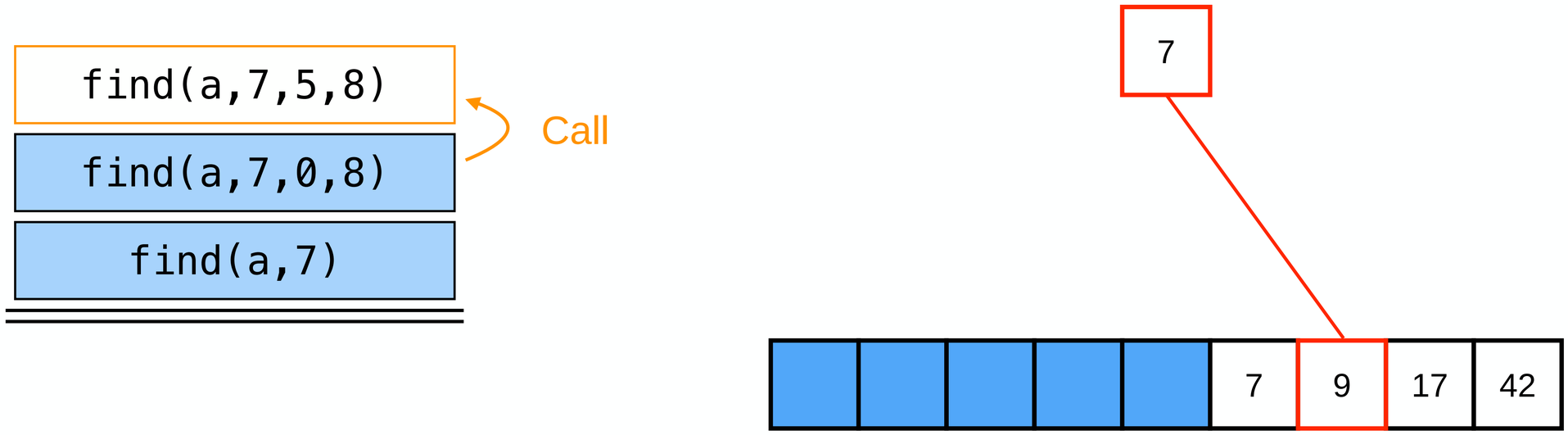
Execution

```
find(a,7)
```

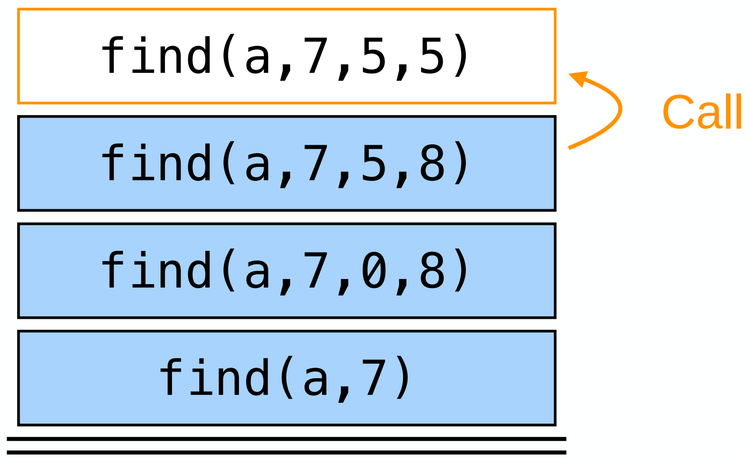
Execution

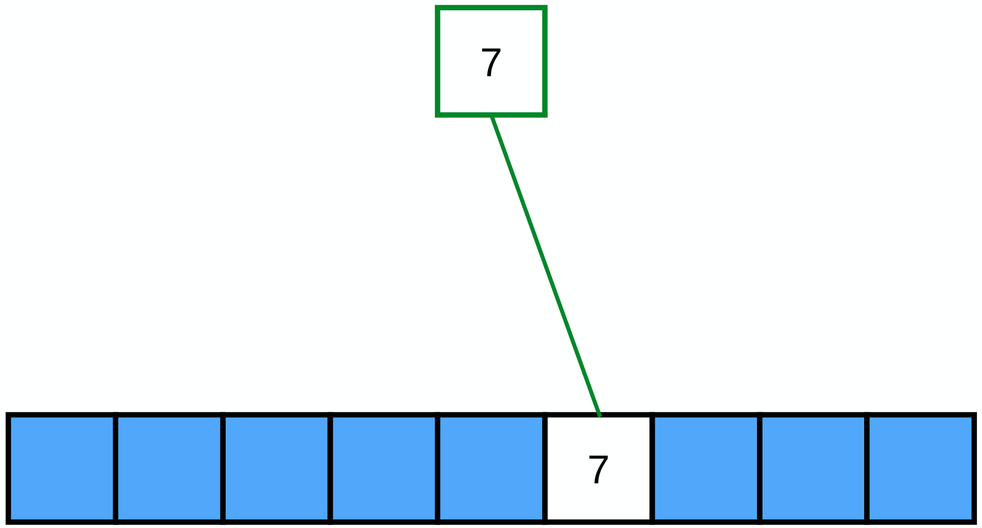


Execution

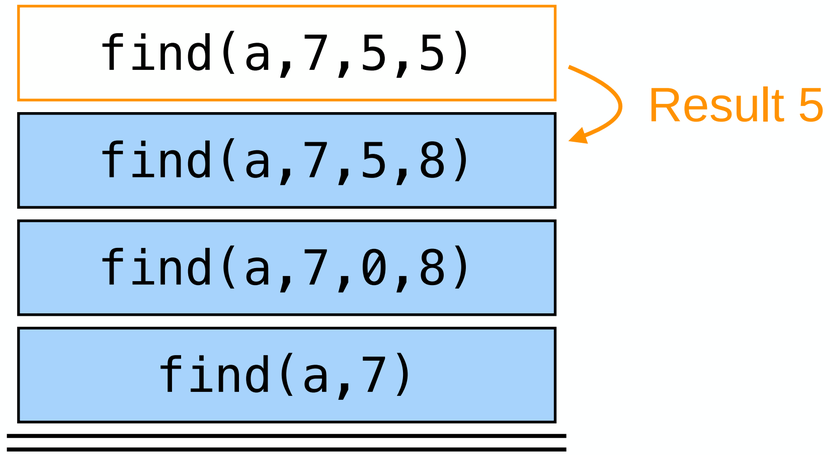


Execution

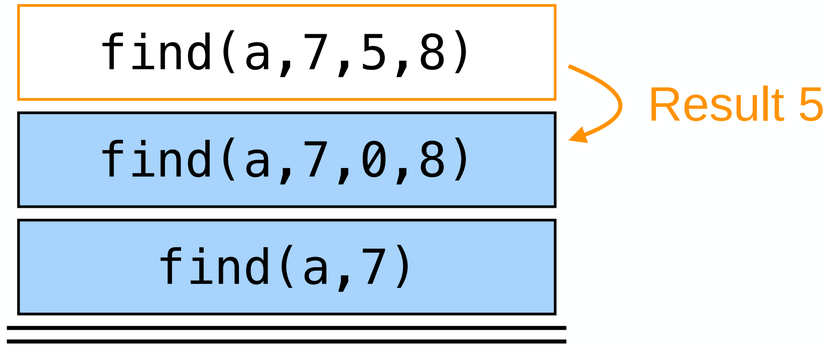




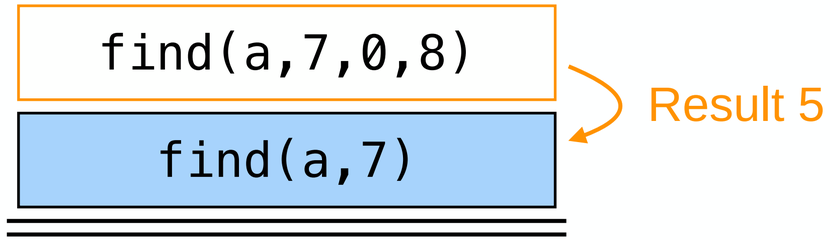
Execution



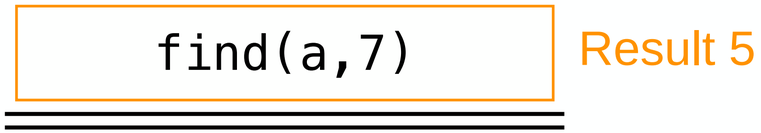
Execution



Execution



Execution



Observation

- The function calls are managed according to the LIFO principle (last in first out)

→ Data structure: **`stack`**

- Only the top/last call is active
- Note: there can be several further inactive calls of the same function at once

Termination

- To **prove** that `find()` terminates, we observe:
  1. If `startIndex` and `endIndex` are the same, `find()` is called for an interval `[n, n]` with **only one** element → the function call terminates directly
  2. If `find()` is called for an interval `[n1, n2]` with more than one element, then
     1. the call either terminates directly (because `element` was found), or
     2. `find()` is called with an interval that is genuinely contained in `[n1, n2]`, or more precisely: even contains at most half of the elements of `[n1, n2]`

→ Similar techniques are also used for other recursive functions

Iterative binary search

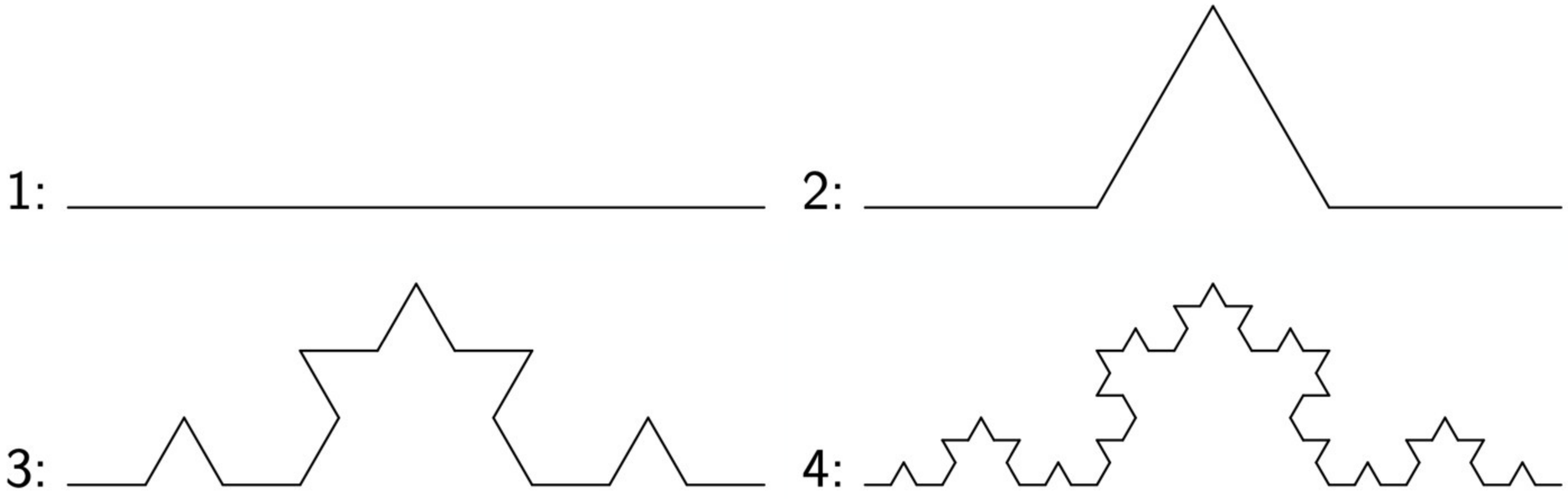
```java
public static int findIterative(int[] array, int element) {
    int startIndex = 0;
    int endIndex = array.length - 1;
    while (true) {
        int middle = (endIndex + startIndex) / 2;
        if (element == array[middle]) {
            return middle;
        } else if (startIndex == endIndex) {
            return -1; // not found
        } else if (element > array[middle]) {
            startIndex = middle + 1;
        } else if (startIndex < middle) {
            endIndex = middle - 1;
        } else {
            return -1; // not found
        }
    }
}
```

Outline

- The principle of recursion
- Example: Towers of Hanoi
- Example: Binary search
- Example: Koch snowflake
- Example: Mergesort

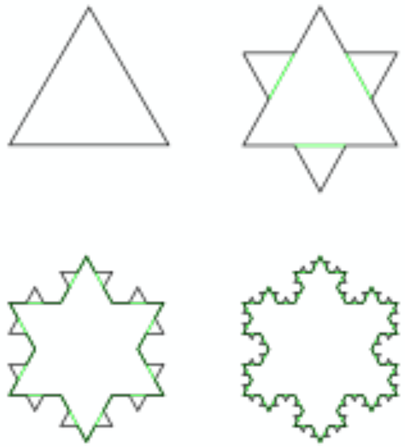
The Koch snowflake curve

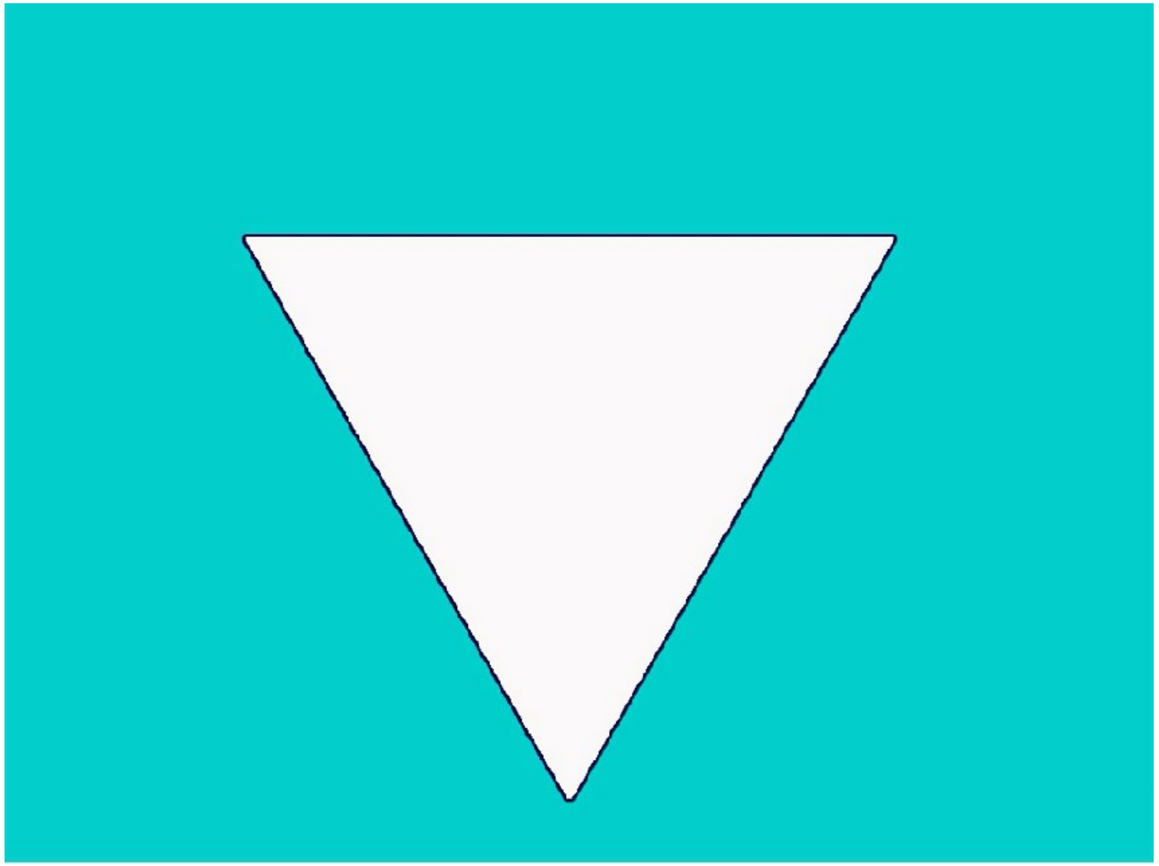
- The Koch curve is a fractal
- It is formed in the following steps
  - Start with a straight line
  - Remove the middle third from each straight line piece
  - Add an equilateral triangle instead



Flake math

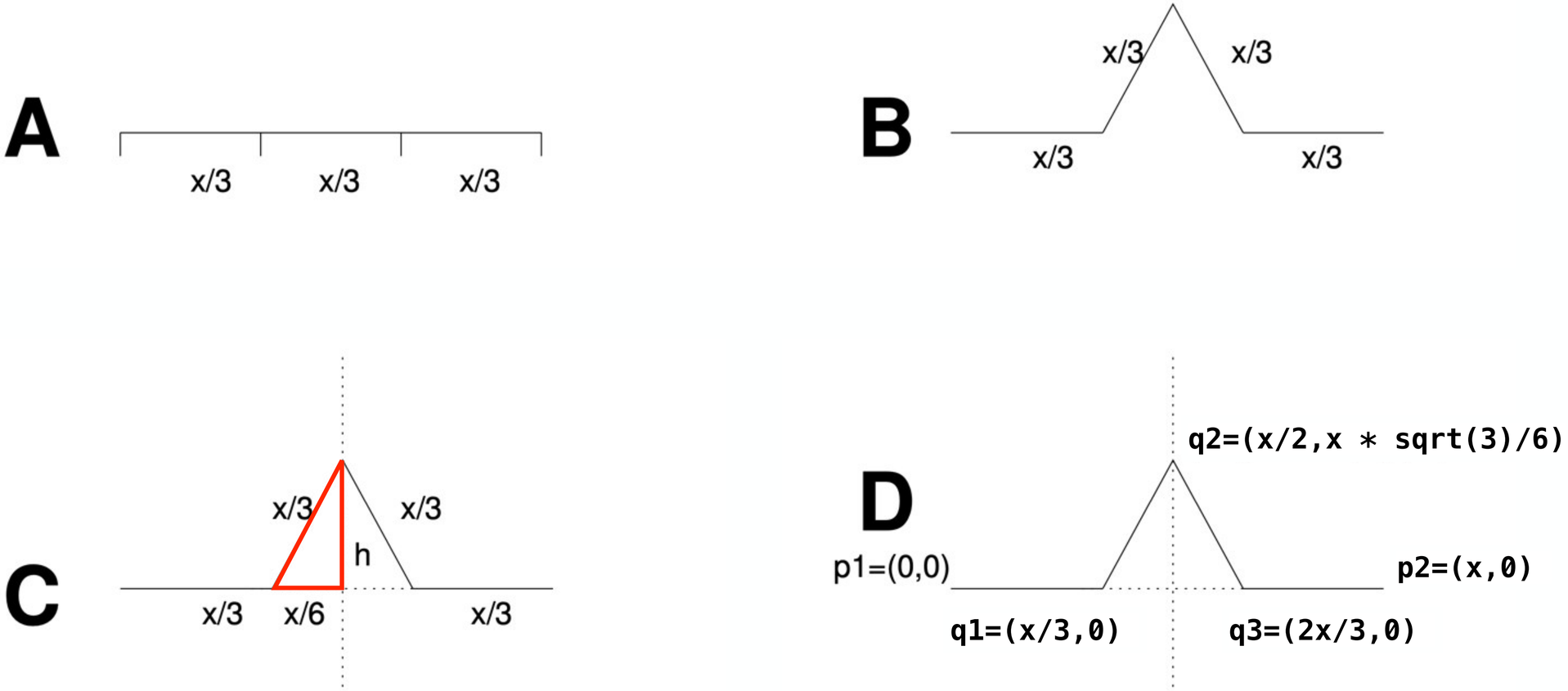
- If you form a Koch curve from each of the three sides of an equilateral triangle, you get the Koch snowflake





Flake formation: horizontal baseline

- From each straight line piece, remove the middle third and put on an equilateral triangle for it



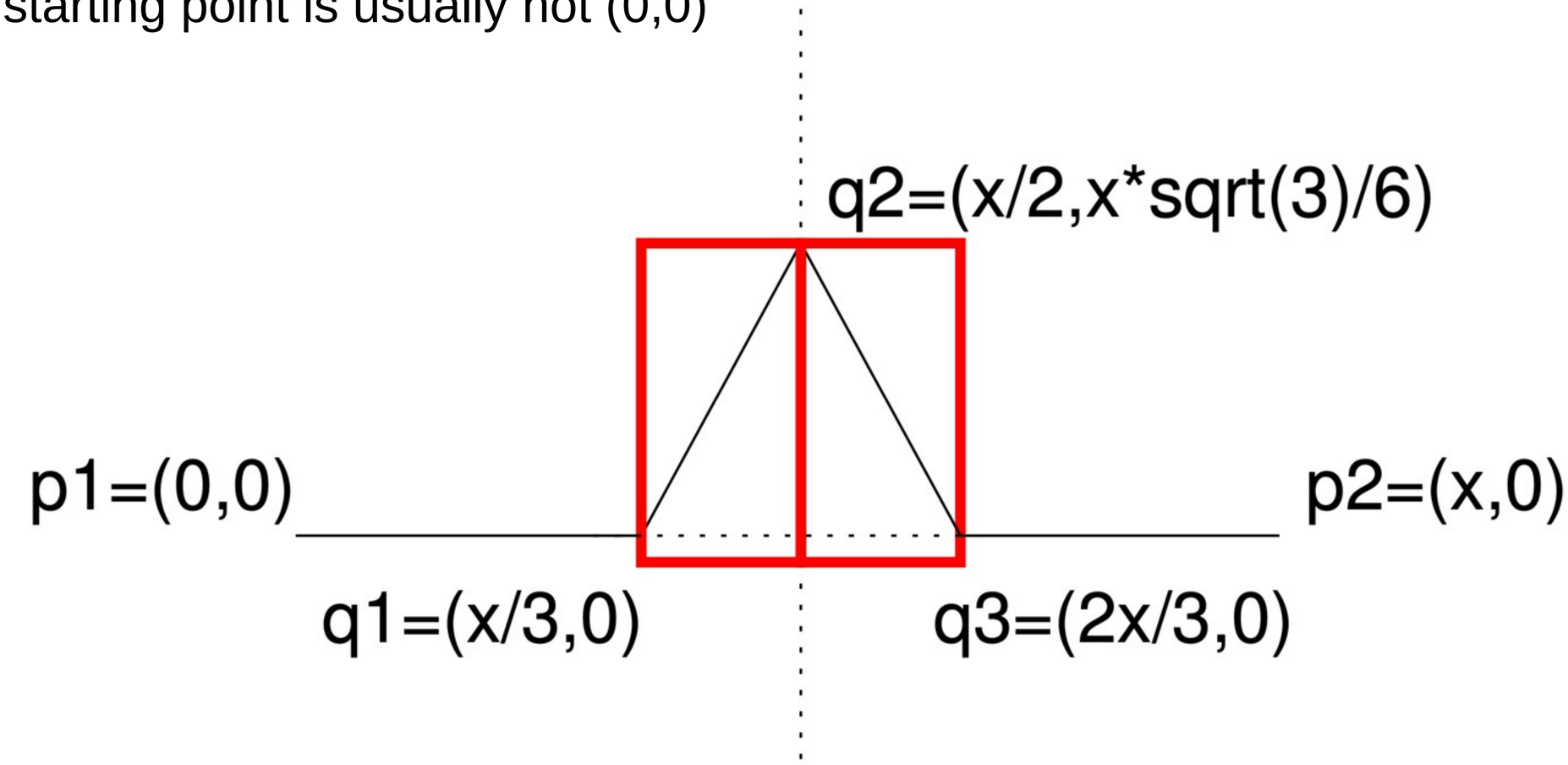
Pythagorean theorem for right triangle

$a^2 + b^2 = c^2$ $\quad$ $x^2/36 + h^2 = x^2/9$

$(x/6)^2 + h^2 = (x/3)^2$ $\quad$ $h = x * \mathrm{sqrt}(3)/6$

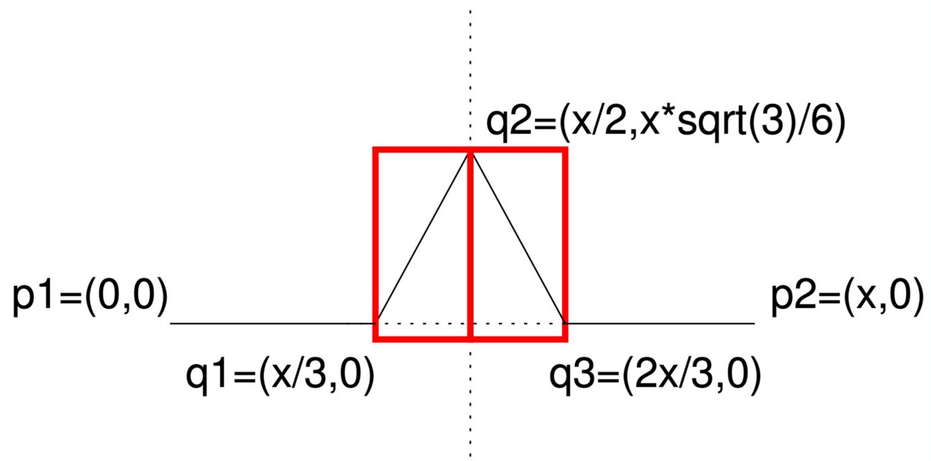
Flake formation: any baseline

- Procedure is applied recursively
  1. The baseline may not be horizontal and
  2. The starting point is usually not (0,0)



Solution: translation and rotation

- Use the simple calculation of the point coordinates
  1. Move the line to the origin by adding the negative coordinates of the first point, i.e. of **`(-x1,-y1)`**
  2. Rotate the shifted line by an angle ϕ so that it is horizontal
  3. Make the simple calculation
  4. Rotate the newly generated part of the flake in the opposite direction using **`-φ`** and shift it back by adding **`(x1, y1)`**



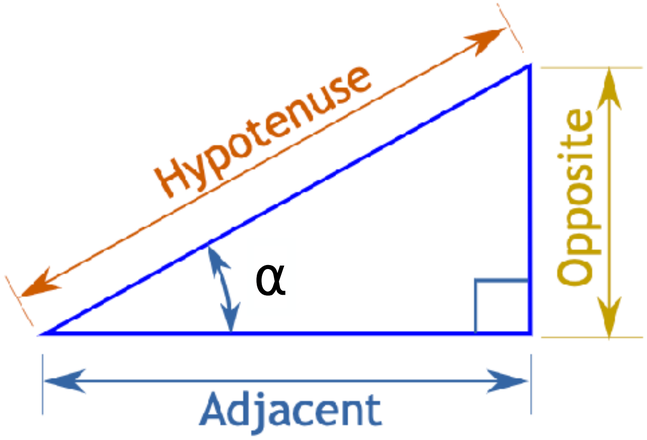
Basic math

- Sine, Cosine and Tangent (often shortened to sin, cos and tan) are each a ratio of sides of a right angled triangle

$$\sin \alpha = \frac{\text{Opposite}}{\text{Hypotenuse}}$$

$$\cos \alpha = \frac{\text{Adjacent}}{\text{Hypotenuse}}$$

$$\tan \alpha = \frac{\text{Opposite}}{\text{Adjacent}}$$

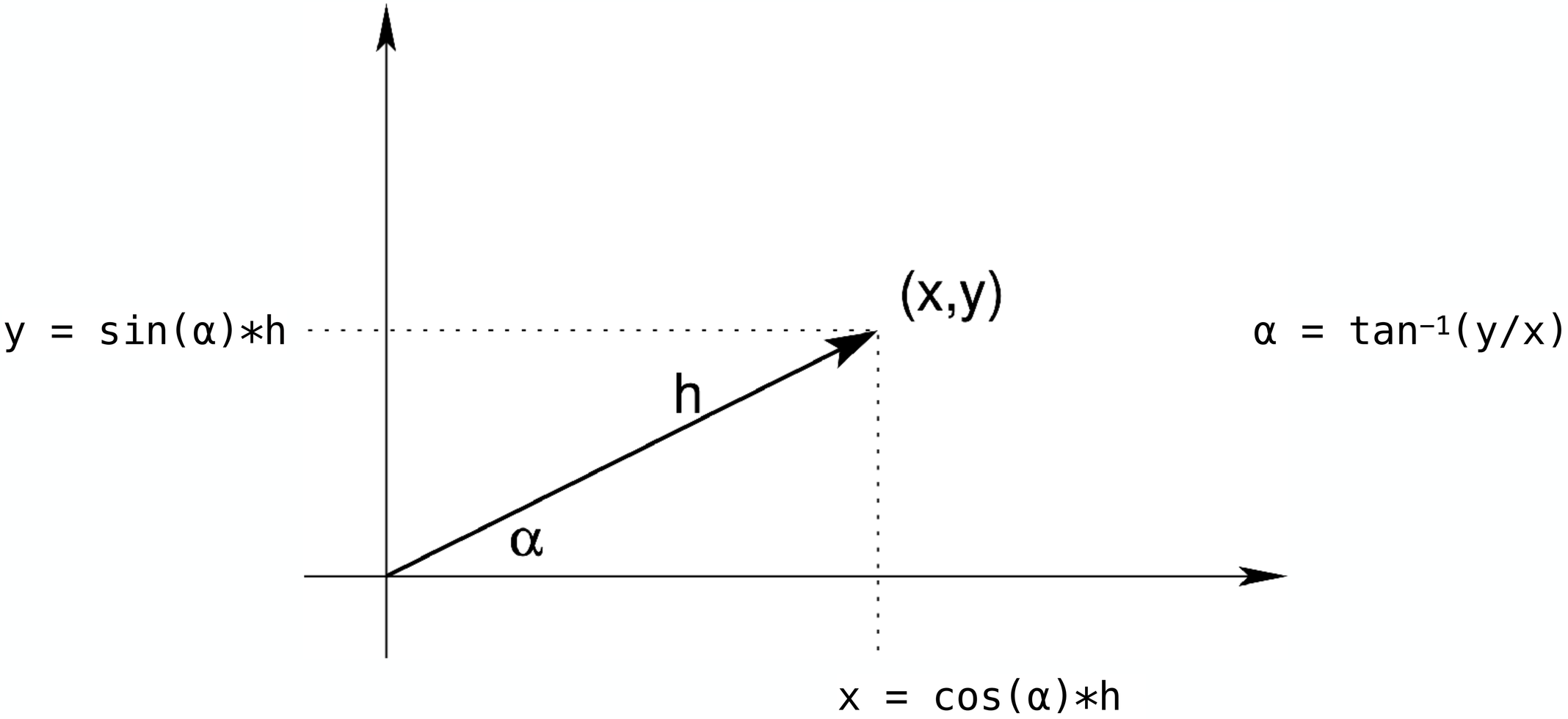


$$\alpha = \tan^{-1}\left(\frac{\text{Opposite}}{\text{Adjacent}}\right)$$

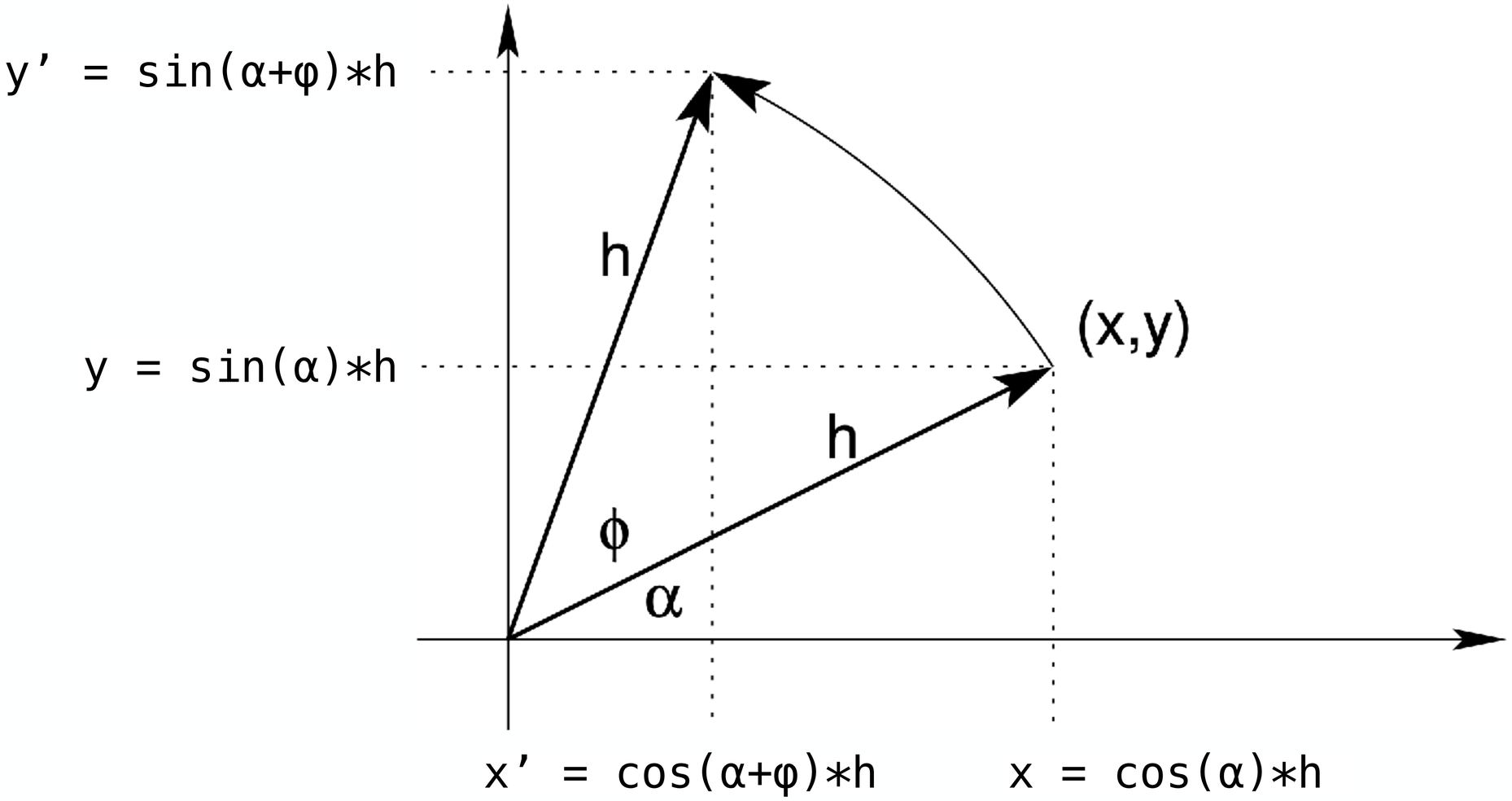
also called `arctan` or **`atan`**

https://www.mathsisfun.com/sine-cosine-tangent.html

Flake math: rotation

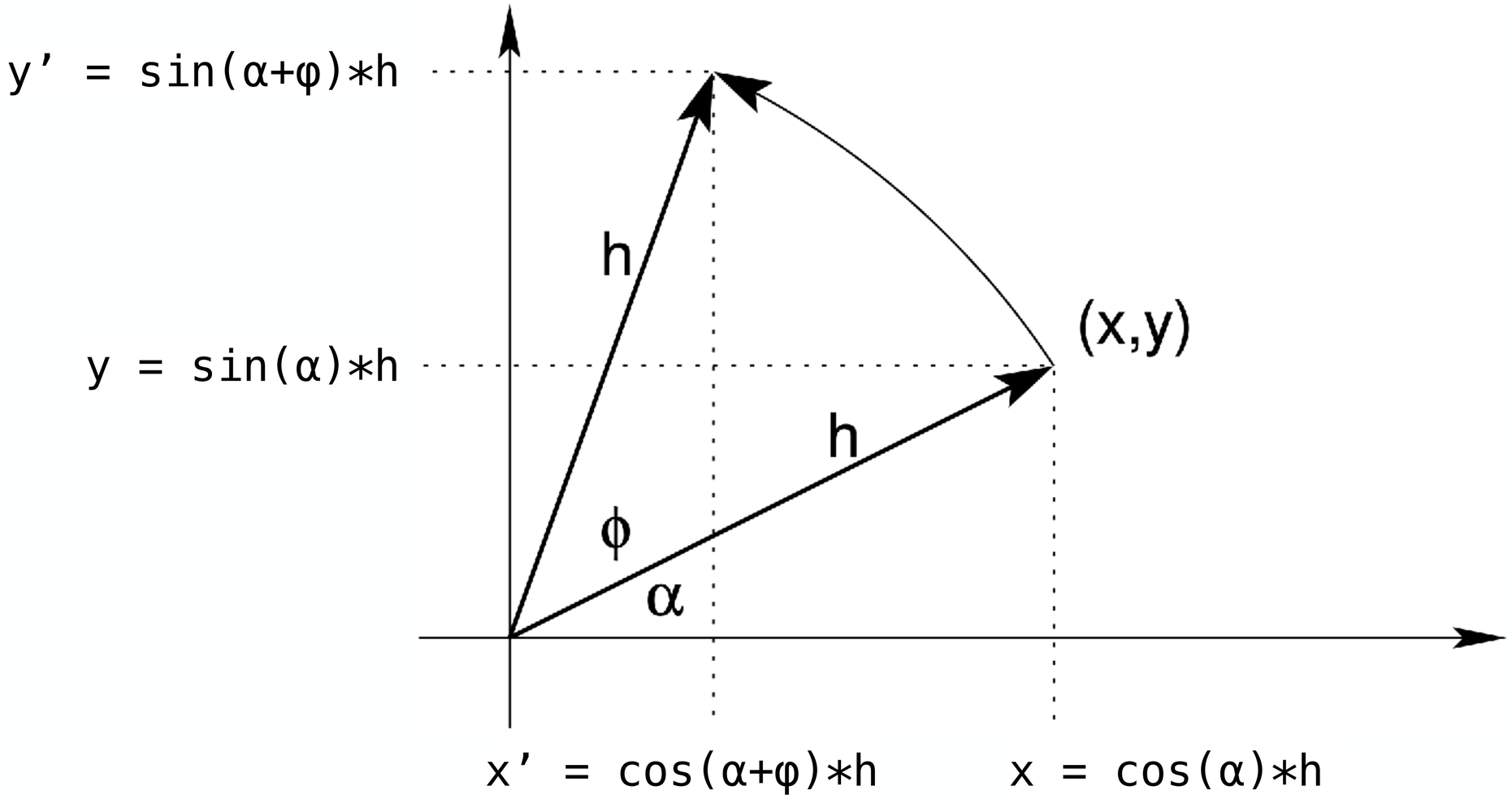


Flake math: rotation



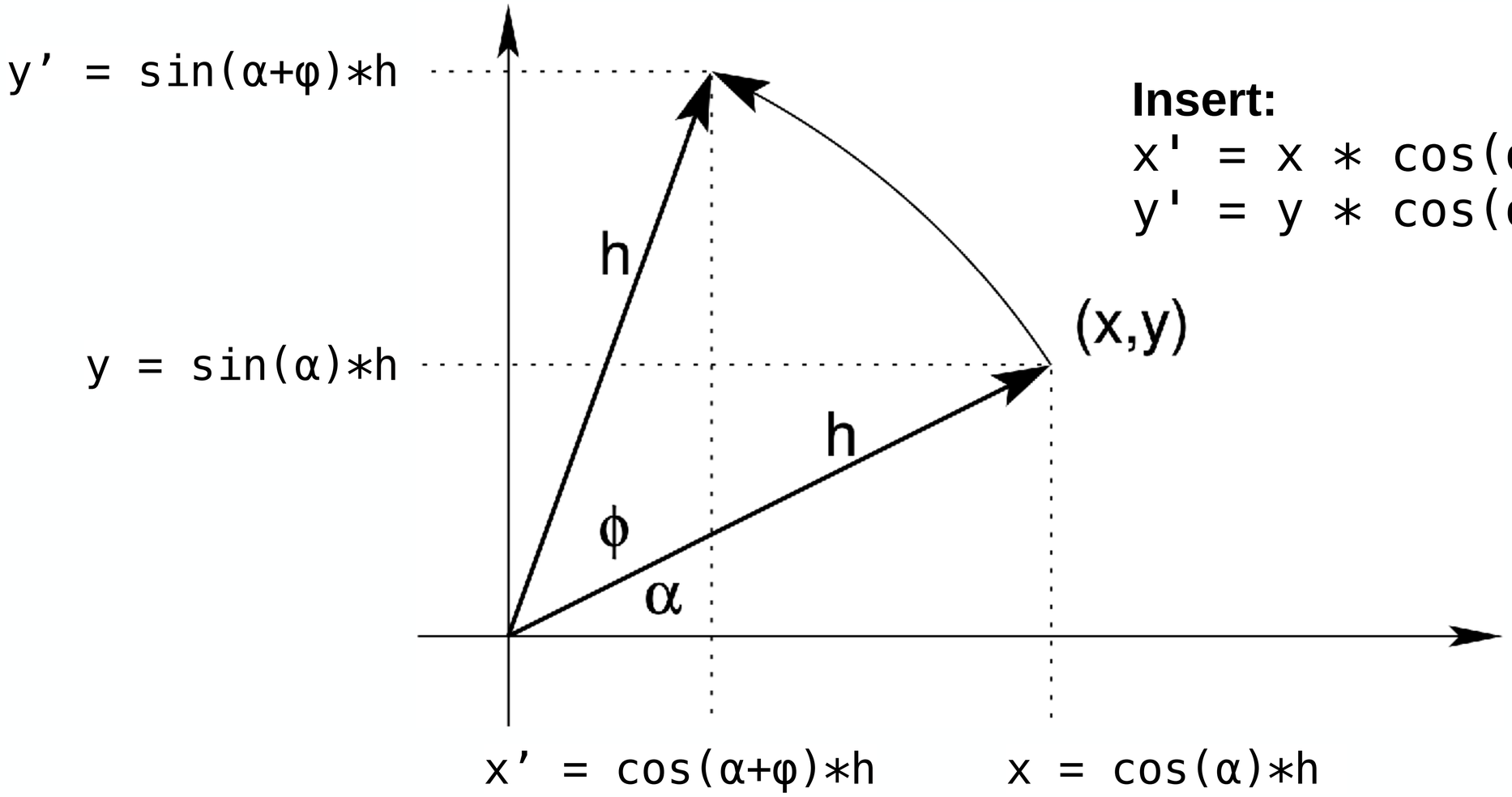
Flake math: rotation

- Addition theorem 1: `sin(α + φ) = sin(α) ∗ cos(φ) + sin(φ) ∗ cos(α)`
- Addition theorem 2: `cos(α + φ) = cos(α) ∗ cos(φ) − sin(φ) ∗ sin(α)`



Flake math: rotation

- Addition theorem 1: `sin(α + φ) = sin(α) ∗ cos(φ) + sin(φ) ∗ cos(α)`
- Addition theorem 2: `cos(α + φ) = cos(α) ∗ cos(φ) − sin(φ) ∗ sin(α)`



Insert:

```
x' = x ∗ cos(φ) − y ∗ sin(φ)
y' = y ∗ cos(φ) + x ∗ sin(φ)
```

Intermediate calculation

- `x' = cos(α + φ)  * h`
- `x' = (cos(α) * cos(φ) – sin(φ) * sin(α)) * h`

Substitute sin(α) = y / h
Substitute cos(α) = x / h

- `x' = (x / h * cos(φ) – sin(φ) * y / h)  * h`
- **`x' =  x * cos(φ)  –  y * sin(φ)`**

- `y' =  sin(α + φ)  * h`
- `y' = (sin(α) * cos(φ) + sin(φ) * cos(α)) * h`

Substitute sin(α) = y / h
Substitute cos(α) = x / h

- `y' = (y / h * cos(φ) + sin(φ) * x / h)  * h`
- **`y' =  y * cos(φ)  +  x * sin(φ)`**

Realization in Java

- Use the class **`Point`**
- Realize the iteration by recursion over the single partial routes
  - Base case: draw the current route
  - Recursion case: calculate the intermediate points of the current path and iterate over all the resulting partial distances
- Calculation of the intermediate points with the help of the operations from the class **`Point`**

Point

```java
public class Point {
    private double x, y;
    Point(double x, double y) { set(x, y); }
    public double getX() { return x; }
    public double getY() { return y; }
    private void set(double x, double y) {
        this.x = x;
        this.y = y;
    }

    Point rotate(double phi) {
        double newX = x * Math.cos(phi) - y * Math.sin(phi);
        double newY = y * Math.cos(phi) + x * Math.sin(phi);
        set(newX, newY);
        return this;
    }

    Point translate(double x, double y) {
        set(this.x + x, this.y + y);
        return this;
    }

    @Override
    public String toString() {
        return "(" + x + "," + y + ')';
    }
}
```

Changes the object on which the method is executed and returns this changed object

Changes the object on which the method is executed and returns this changed object

Flake

```java
import java.util.*;

public class Flake {

    private final List<Point> points = new ArrayList<>();

    Flake(Point p1, Point p2) {
        points.add(p1);
        points.add(p2);
    }

    public List<Point> getPoints() {
        return points;
    }

    //...

    @Override
    public String toString() {
        return "Flake{" + points + '}';
    }
}
```

Flake.recursiveSnow

```java
public void recursiveSnow(Point startPoint, Point endPoint, int depth) {
    if (depth > 0) {
        double shiftX = startPoint.getX(), shiftY = startPoint.getY();
        endPoint.translate(-shiftX, -shiftY);

        double angle = Math.atan2(endPoint.getY(), endPoint.getX());
        endPoint.rotate(-angle);

        Point helperPoint = new Point(endPoint.getX() / 3.0, 0.0);
        Point mid1 = helperPoint.rotate(angle).translate(shiftX, shiftY);
        helperPoint = new Point(endPoint.getX() / 2.0, endPoint.getX() / 6.0 * Math.sqrt(3.0));
        Point mid2 = helperPoint.rotate(angle).translate(shiftX, shiftY);
        helperPoint = new Point(2.0 * endPoint.getX() / 3.0, 0.0);
        Point mid3 = helperPoint.rotate(angle).translate(shiftX, shiftY);

        points.addAll(points.indexOf(startPoint) + 1, List.of(mid1, mid2, mid3));

        endPoint.rotate(angle).translate(shiftX, shiftY);

        recursiveSnow(startPoint, mid1, depth - 1);
        recursiveSnow(mid1, mid2, depth - 1);
        recursiveSnow(mid2, mid3, depth - 1);
        recursiveSnow(mid3, endPoint, depth - 1);
    }
}
```

Flake.recursiveSnow (with comments)

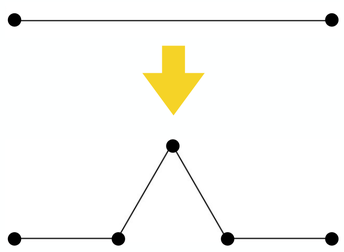
Important: terminating condition

```
public void recursiveSnow(Point startPoint, Point endPoint, int depth) {
    if (depth > 0) {
        double shiftX = startPoint.getX(), shiftY = startPoint.getY();
        endPoint.translate(-shiftX, -shiftY);

        double angle = Math.atan2(endPoint.getY(), endPoint.getX());
        endPoint.rotate(-angle);

        Point helperPoint = new Point(endPoint.getX() / 3.0, 0.0);
        Point mid1 = helperPoint.rotate(angle).translate(shiftX, shiftY);
        helperPoint = new Point(endPoint.getX() / 2.0, endPoint.getX() / 6.0 * Math.sqrt(3.0));
        Point mid2 = helperPoint.rotate(angle).translate(shiftX, shiftY);
        helperPoint = new Point(2.0 * endPoint.getX() / 3.0, 0.0);
        Point mid3 = helperPoint.rotate(angle).translate(shiftX, shiftY);

        points.addAll(points.indexOf(startPoint) + 1, List.of(mid1, mid2, mid3));

        endPoint.rotate(angle).translate(shiftX, shiftY);

        recursiveSnow(startPoint, mid1, depth - 1);
        recursiveSnow(mid1, mid2, depth - 1);
        recursiveSnow(mid2, mid3, depth - 1);
        recursiveSnow(mid3, endPoint, depth - 1);
    }
}
```

returns the angle of the point

Rotate end point to a horizontal line

Compute three new **mid points** using **rotate** and then **translate**

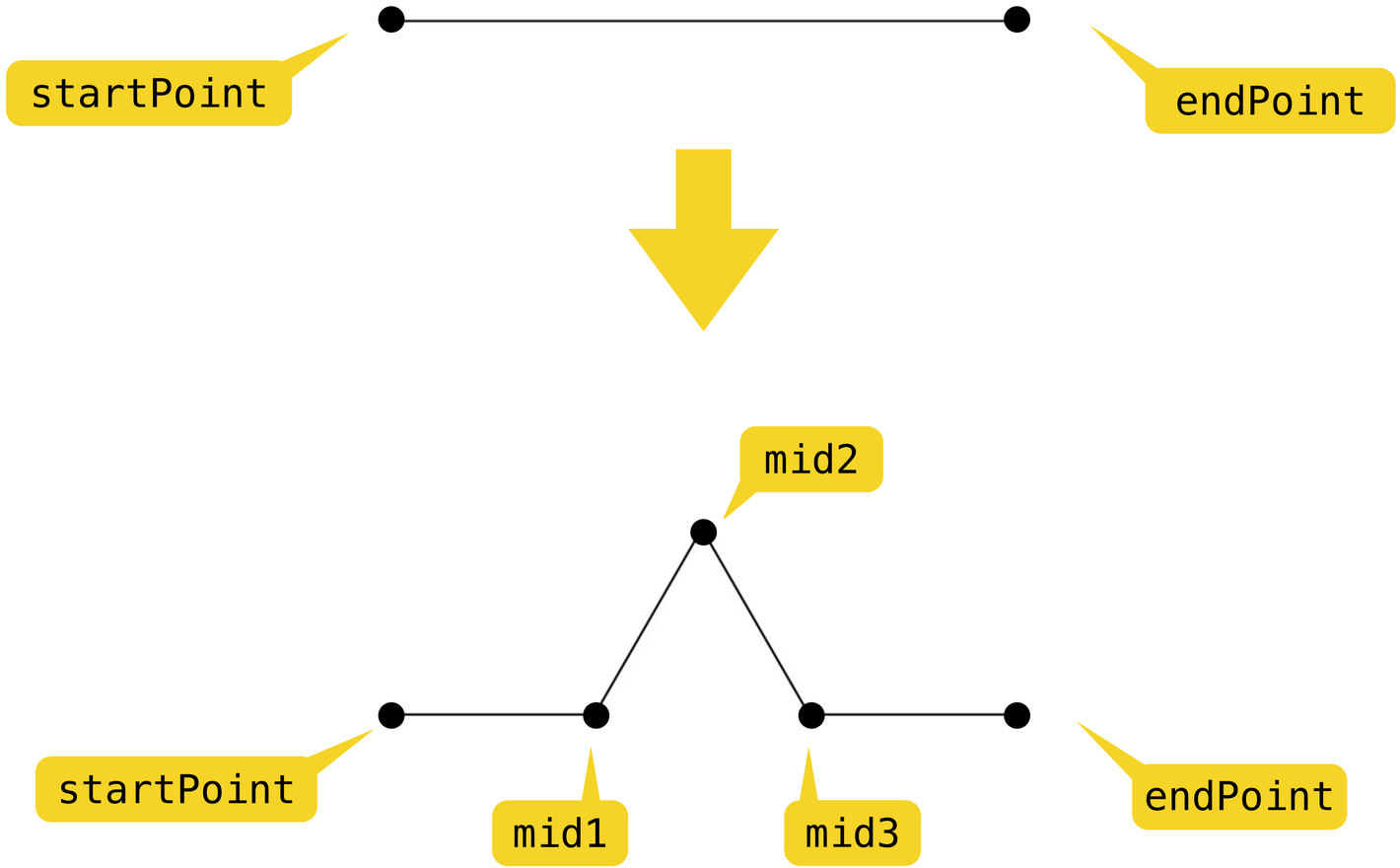


Create new polygon: add 3 mid points after start point

Undo translation and rotation

Recursion: create new flake for each segment

Calculation of mid points



Flake.snow (iterative)

```java
public void snow(int depth) {
    for (int i = 0; i < depth; i++) {
        List<Point> newPoints = new ArrayList<>();
        Point endPoint = null;
        for (int j = 0; j < points.size() - 1; j++) {
            Point startPoint = points.get(j);
            endPoint = points.get(j + 1);
            double shiftX = startPoint.getX(), shiftY = startPoint.getY();
            endPoint.translate(-shiftX, -shiftY);
            // rotate to horizontal
            double angle = Math.atan2(endPoint.getY(), endPoint.getX());
            endPoint.rotate(-angle);
            // compute three new mid points using rotate and then translate
            Point helperPoint = new Point(endPoint.getX() / 3.0, 0.0);
            Point mid1 = helperPoint.rotate(angle).translate(shiftX, shiftY);
            helperPoint = new Point(endPoint.getX() / 2.0, endPoint.getX() / 6.0 * Math.sqrt(3.0));
            Point mid2 = helperPoint.rotate(angle).translate(shiftX, shiftY);
            helperPoint = new Point(2.0 * endPoint.getX() / 3.0, 0.0);
            Point mid3 = helperPoint.rotate(angle).translate(shiftX, shiftY);
            // create new polygon: add 3 mid points after start point
            newPoints.addAll(List.of(startPoint, mid1, mid2, mid3));
            // undo translation and rotation
            endPoint.rotate(angle).translate(shiftX, shiftY);
        }
        newPoints.add(endPoint);
        points.clear();
        points.addAll(newPoints);
    }
}
```

Flake.snow (iterative, with comments)

```java
public void snow(int depth) {
    for (int i = 0; i < depth; i++) {
        List<Point> newPoints = new ArrayList<>();
        Point endPoint = null;
        for (int j = 0; j < points.size() - 1; j++) {
            Point startPoint = points.get(j);
            endPoint = points.get(j + 1);
            double shiftX = startPoint.getX(), shiftY = startPoint.getY();
            endPoint.translate(-shiftX, -shiftY);
            // rotate to horizontal
            double angle = Math.atan2(endPoint.getY(), endPoint.getX());
            endPoint.rotate(-angle);
            // compute three new mid points using rotate and then translate
            Point helperPoint = new Point(endPoint.getX() / 3.0, 0.0);
            Point mid1 = helperPoint.rotate(angle).translate(shiftX, shiftY);
            helperPoint = new Point(endPoint.getX() / 2.0, endPoint.getX() / 6.0 * Math.sqrt(3.0));
            Point mid2 = helperPoint.rotate(angle).translate(shiftX, shiftY);
            helperPoint = new Point(2.0 * endPoint.getX() / 3.0, 0.0);
            Point mid3 = helperPoint.rotate(angle).translate(shiftX, shiftY);
            // create new polygon: add 3 mid points after start point
            newPoints.addAll(List.of(startPoint, mid1, mid2, mid3));
            // undo translation and rotation
            endPoint.rotate(angle).translate(shiftX, shiftY);
        }
        newPoints.add(endPoint);
        points.clear();
        points.addAll(newPoints);
    }
}
```

Use iteration instead of recursion

Iterate through the array and get the current **`startPoint`** and **`endPoint`** for each segment

Do not add the **`endPoint`**, it will be processed and added in the next iteration (inner for loop)

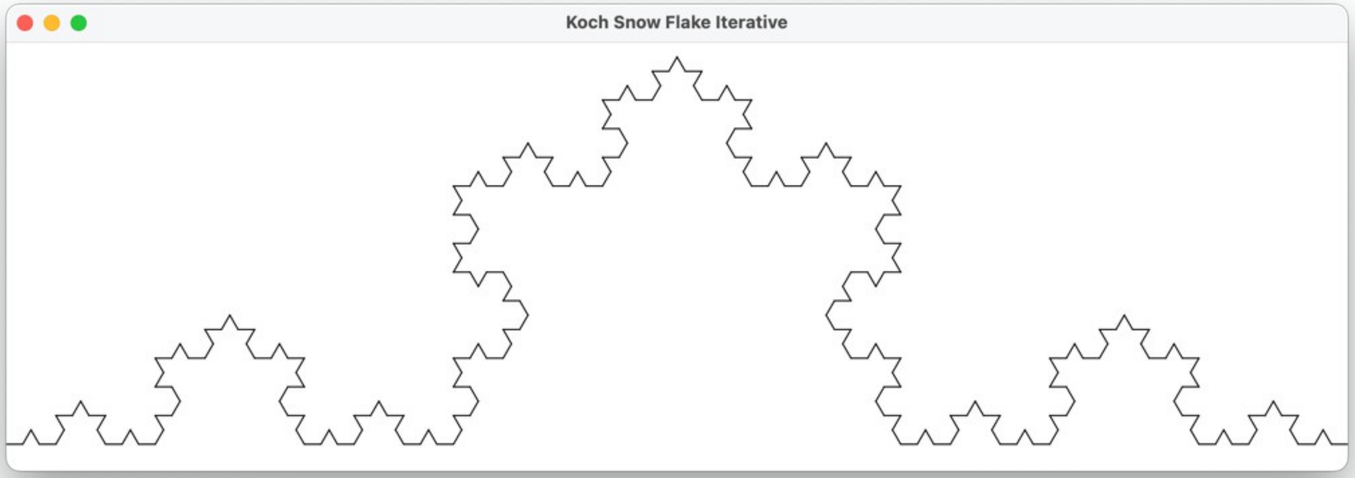
Add the **`endPoint`** (once for the whole iteration) and use all new points from now on

Usage

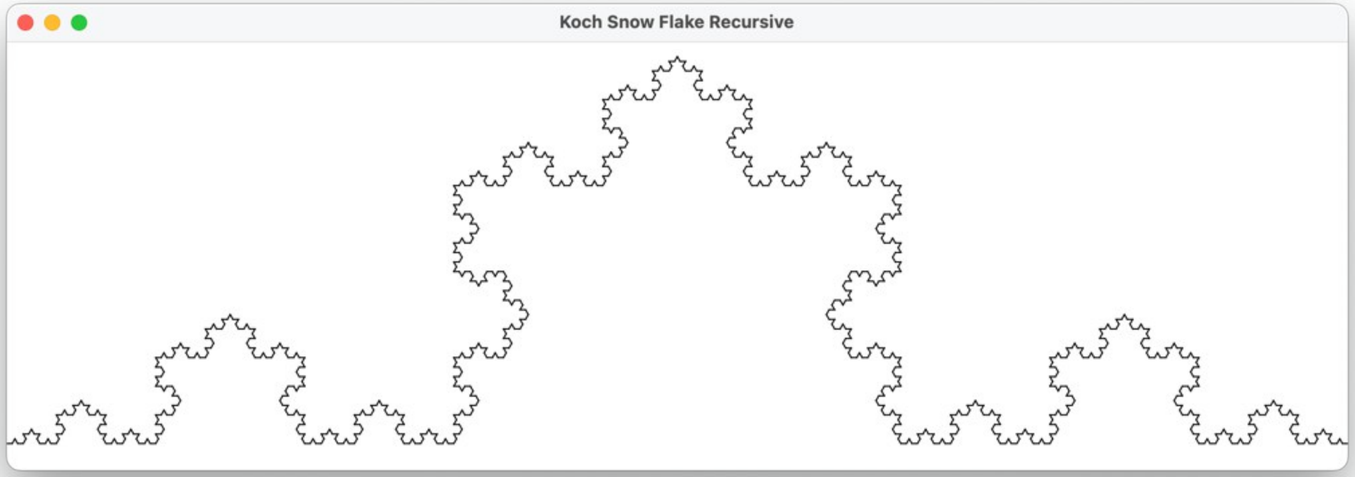
```java
package de.tum.in.ase;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Line;
import javafx.stage.Stage;

public class KochSnowflake extends Application {

    @Override
    public void start(Stage primaryStage) {
        Point startPoint = new Point(0, 0);
        Point endPoint = new Point(1000, 0);

        Flake recursiveFlake = new Flake(startPoint, endPoint);
        recursiveFlake.recursiveSnow(startPoint, endPoint, 5);

        Flake iterativeFlake = new Flake(startPoint, endPoint);
        iterativeFlake.snow(4);

        Pane pane = new Pane();
        for (int i = 0; i < iterativeFlake.getPoints().size() - 1; i++) {
            Point p1 = iterativeFlake.getPoints().get(i);
            Point p2 = iterativeFlake.getPoints().get(i + 1);
            pane.getChildren().add(new Line(p1.getX(), 300 - p1.getY(), p2.getX(), 300 - p2.getY()));
        }
        primaryStage.setScene(new Scene(pane, 1000, 320));
        primaryStage.setTitle("Koch Snow Flake Iterative");
        primaryStage.show();
    }
}
```

Output

Depth: 4



Depth: 5



You can find the whole example on https://github.com/ls1intum/Snow-Flake-Exercise

W12E03 - N-th power of 10

Start exercise

Easy

Not started yet.

Due by tonight

5 min

2 pts

- **Problem statement**: Calculate the **`n-th power of 10`** using recursion
- Examples
  - `powerOf10(0) = 1`
  - `powerOf10(1) = 10`
  - `powerOf10(5) = 100.000`

Outline

- The principle of recursion
- Example: Towers of Hanoi
- Example: Binary search
- Example: Koch snowflake
- Example: Mergesort

Review: **mergesort** step 3 (iterative)

```java
public static <T extends Comparable<T>> List<T> merge(List<T> a, List<T> b) {
    List<T> result = new ArrayList<>();
    int i = 0;  // index for input list a
    int j = 0;  // index for input list b

    while (i < a.size() && j < b.size()) {
        if (a.get(i).compareTo(b.get(j)) < 0) {
            result.add(a.get(i++));
        } else {
            result.add(b.get(j++));
        }
    }

    while (i < a.size()) {
        result.add(a.get(i++));
    }
    while (j < b.size()) {
        result.add(b.get(j++));
    }

    return result;
}
```

Take the smaller element

Append all remaining elements of a into result

Append all remaining elements of b into result

Mergesort recursive: merge two lists

- If one of the two lists **`a`** and **`b`** is empty, output the other
- Otherwise, there is a first (smallest) element in each of the two lists
- From these two elements, take the smallest one
- Behind it, put the list by merging the remaining elements

Code: merge two lists

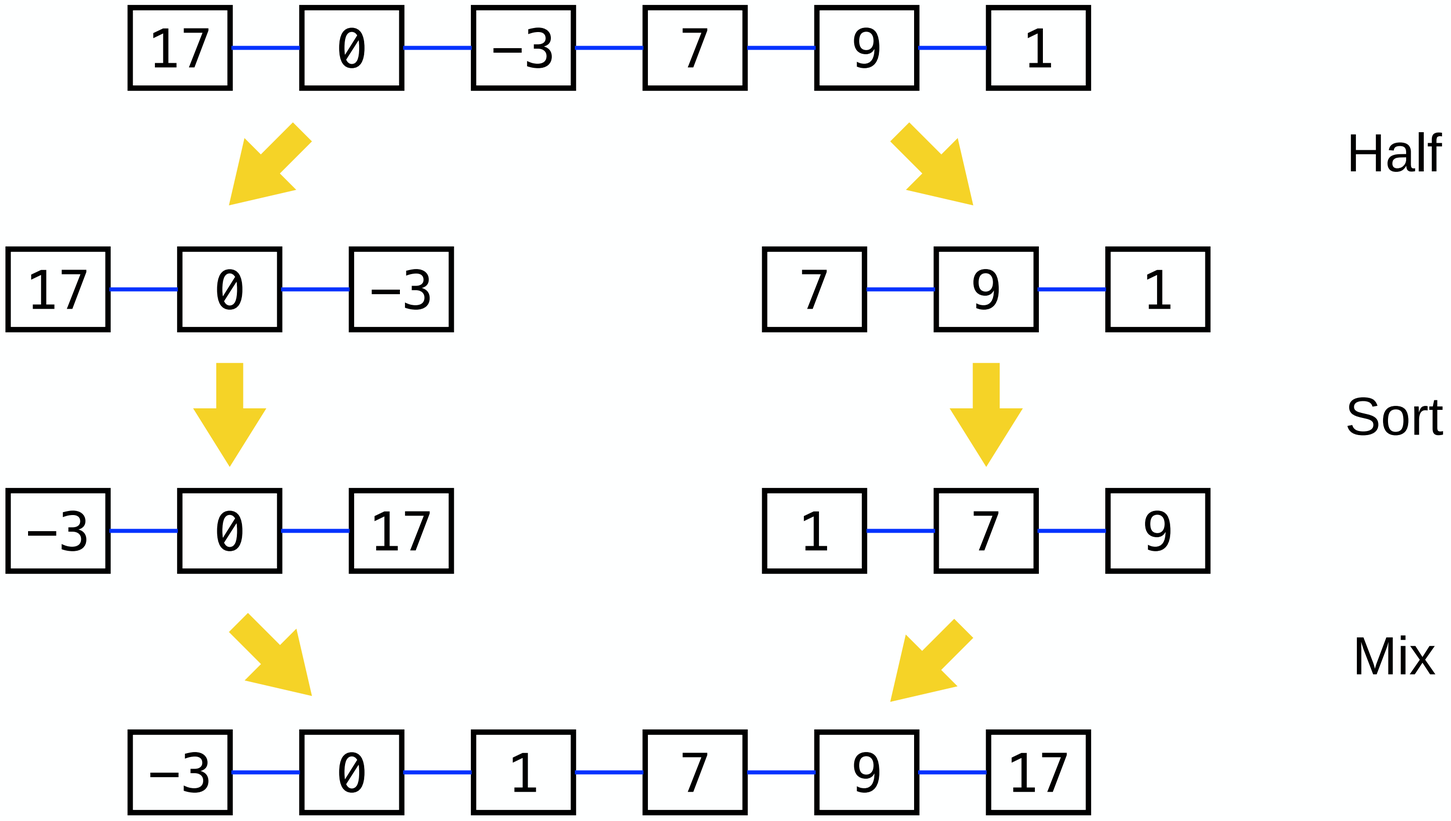
```
public static <T extends Comparable<T>> List<T> mergeRecursive(List<T> a, List<T> b) {
    if (a == null || a.isEmpty()) {
        return b;
    }
    if (b == null|| b.isEmpty()) {
        return a;
    }
    List<T> newList = new ArrayList<>();
    if (a.get(0).compareTo(b.get(0)) < 0) {
        newList.add(a.remove(0));
    } else {
        newList.add(b.remove(0));
    }
    newList.addAll(mergeRecursive(a, b));
    return newList;
}
```

Terminating condition

Direct recursive method calls

Sort by mixing

- Divide list to be sorted into two partial lists
- Sort each half separately
- Mix the results
- Example



Code: half

```
public static <T extends Comparable<T>> List<T> leftHalf(List<T> list) {
    List<T> leftHalf = new ArrayList<>();
    for (int i = 0; i < list.size() / 2; i++) {
        leftHalf.add(list.get(i));
    }
    return leftHalf;
}

public static <T extends Comparable<T>> List<T> rightHalf(List<T> list) {
    List<T> rightHalf = new ArrayList<>();
    for (int i = list.size() / 2; i < list.size(); i++) {
        rightHalf.add(list.get(i));
    }
    return rightHalf;
}
```

You can find the whole code including usage examples on https://gist.github.com/krusche/d18e381275016e2e74e98157831d78ac

Code: sort

```
public static <T extends Comparable<T>> List<T> sort(List<T> list) {
    if (list == null || list.size() <= 1) {
        return list;
    }
    List<T> left = leftHalf(list);
    List<T> right = rightHalf(list);
    left = sort(left);
    right = sort(right);
    return mergeRecursive(left, right);
}
```

Terminating condition

Direct recursive method calls

You can find the whole code including usage examples on https://gist.github.com/krusche/d18e381275016e2e74e98157831d78ac

W12E04 - Fibonacci Sequence

Start exercise

Easy

Not started yet.

Due by tonight

10 min

3 pts

- **Problem statement**
  - Fibonacci sequence: `0 1 1 2 3 5 8 13 21 34 55 ...`
  - The **`n-th`** element is the sum of the two previous elements
  - Calculate the **`n-th`** element of the fibonacci sequence using a recursive function
  - Examples
    - `fibonacci(2)  = 1`
    - `fibonacci(3)  = 2`
    - `fibonacci(7)  = 13`
    - `fibonacci(10) = 55`
  - Optimize the calculation with an iterative version

Fibonacci performance characteristics

- Exponential complexity: $O(2^n)$
- Bad implementation: lots of repeated and unnecessary calculations!
- **`fibonacci(50)`** cannot be computed in a reasonable amount of time using the recursive function

```
                          fib(5)
                     /                 \
               fib(4)                 fib(3)
             /        \               /       \
         fib(3)      fib(2)         fib(2)   fib(1)
        /    \       /    \        /      \
  fib(2)   fib(1)  fib(1) fib(0) fib(1) fib(0)
  /     \
fib(1) fib(0)
```

Iterative solution

- Complexity: **`O(n)`**

```java
public static long fibonacciIterative(int n) {

    long[] fibNumbers = new long[n + 2];

    fibNumbers[0] = 0;
    fibNumbers[1] = 1;

    for (int i = 2; i <= n; i++) {

        fibNumbers[i] = fibNumbers[i - 1] + fibNumbers[i - 2];
    }

    return fibNumbers[n];
}
```

Declare an array to store fibonacci numbers (2 additional elements to handle the cases **`n = 0`** and **`n = 1`**)

0th and **1st** number of the series are **0** and **1**

Add the previous **2 numbers** in the series and store it

Next steps

- **Tutor group** exercise
  - T12E01 - Into the Seal Matrix (1999)
- **Homework** exercise
  - H12E01 - TUM Seal Triangle
- **Project work**
  - Implement the game
- Read the following articles
  - https://www.baeldung.com/java-recursion
  - https://introcs.cs.princeton.edu/java/23recursion

→ Due by **Wednesday, January 21, 13:00**

Summary

- Recursion is the technique of **invoking a function within itself**
- It provides a way to **break complicated problems down into simple ones** which are easier to solve
- Recursion may be **difficult to understand**
- The best way to figure out how it works is to **experiment with it in multiple examples**
- There are two main requirements of a recursive function
  - **A terminating condition:** the function returns a (simple) value when a certain condition is satisfied, without a further recursive call
  - **The recursive call:** the function calls itself with an input which is a step closer to the terminating condition
- Sometimes, iteration is the significantly faster approach (e.g. fibonacci)

References

- https://www.w3schools.com/java/java_recursion.asp
- https://www.baeldung.com/java-recursion
- https://www.geeksforgeeks.org/recursion-in-java
- https://www.geeksforgeeks.org/merge-sort
- https://introcs.cs.princeton.edu/java/23recursion
- https://books.trinket.io/thinkjava2/chapter8.html
- https://codingbat.com/java/Recursion-1
- https://codingbat.com/java/Recursion-2
- https://mathworld.wolfram.com/KochSnowflake.html
- https://en.wikipedia.org/wiki/Tower_of_Hanoi
- https://www.baeldung.com/java-binary-search